

# Containerization

# Learning Objectives

By the end of this session, you should be able to:

- Identify the necessity of containerization in model deployment and its advantages over virtualization.
- Describe the benefits of Docker in ensuring consistent deployments using containers.
- Explain the structure and functionality of a REST API for deploying trained ML models.
- Outline the scalability, integration, and management aspects of ML solutions using Docker containers and REST APIs.

# Agenda

In this session, we'll discuss:

- Challenges with Model Deployment
- Need for Virtualization and Containerization
- Images in Containerization
- Dockerfile
- Containerization Platforms and Docker
- Need for REST API
- REST API Frameworks

# Video Break



# Importance of Model Deployment

**Deployment** solves critical challenges associated with **making ML models accessible, scalable, and efficient.**

# Importance of Model Deployment

**Deployment** solves critical challenges associated with **making ML models accessible, scalable, and efficient.**

- No need to **open and run a Python notebook** for every prediction.

# Importance of Model Deployment

**Deployment** solves critical challenges associated with **making ML models accessible, scalable, and efficient.**

- No need to **open and run a Python notebook** for every prediction.
- No need to **process data manually** in the notebook when new data comes in.

# Importance of Model Deployment

**Deployment** solves critical challenges associated with **making ML models accessible, scalable, and efficient.**

- No need to **open and run a Python notebook** for every prediction.
- No need to **process data manually** in the notebook when new data comes in.
- No **long waits for predictions** while the entire notebook executes.

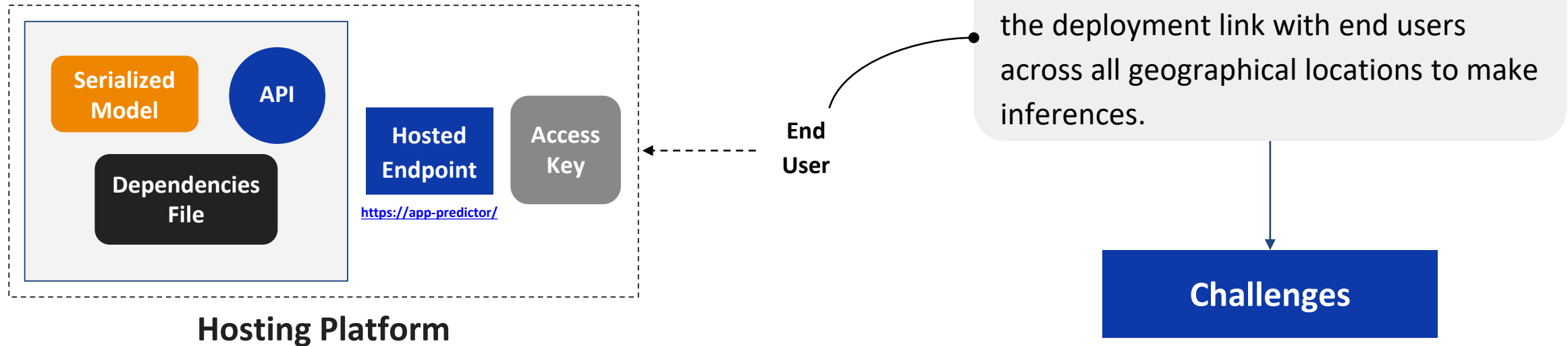
# Importance of Model Deployment

**Deployment** solves critical challenges associated with **making ML models accessible, scalable, and efficient.**

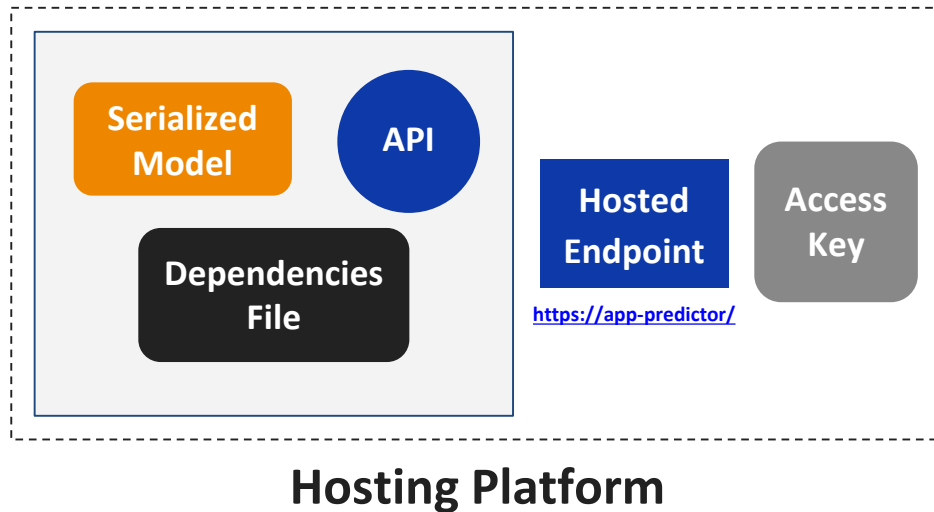
- No need to **open and run a Python notebook** for every prediction.
- No need to **process data manually** in the notebook when new data comes in.
- No **long waits for predictions** while the entire notebook executes.

By **hosting the serialized model** on a platform (preferably as an app) and **setting it up for inference**, these issues are eliminated.

# Challenges with Model Deployment



# Challenges with Model Deployment



Once deployment is successful, share the deployment link with end users across all geographical locations to make inferences.

End User

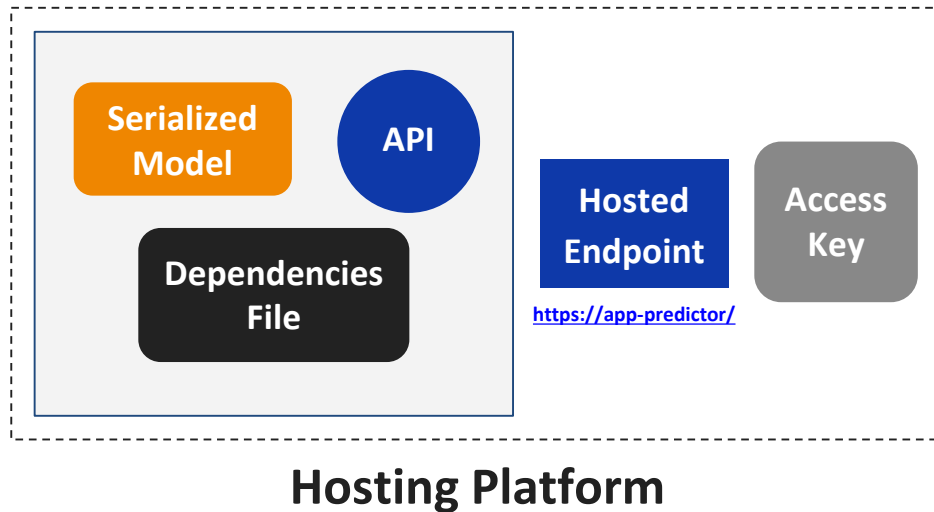
**Challenges**

**Performance**

**Scalability**

Predictions depend on a single hosted app, increasing response times and risking service disruptions if it faces connectivity issues.

# Challenges with Model Deployment



Once deployment is successful, share the deployment link with end users across all geographical locations to make inferences.

End User

**Challenges**

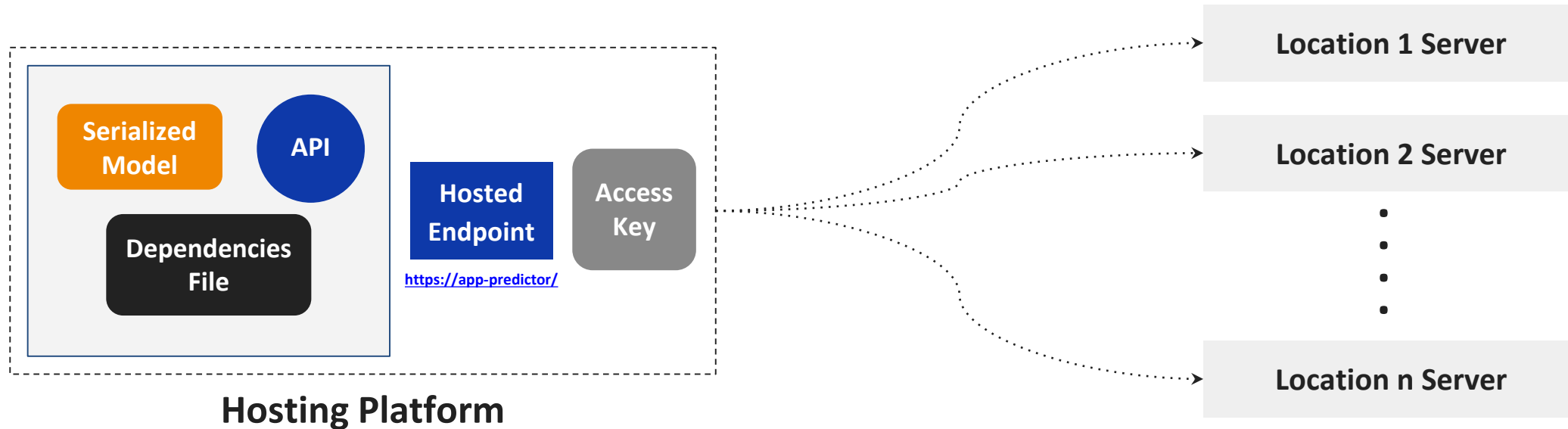
**Performance**

**Scalability**

App crashes when multiple requests are made at the same point of time.

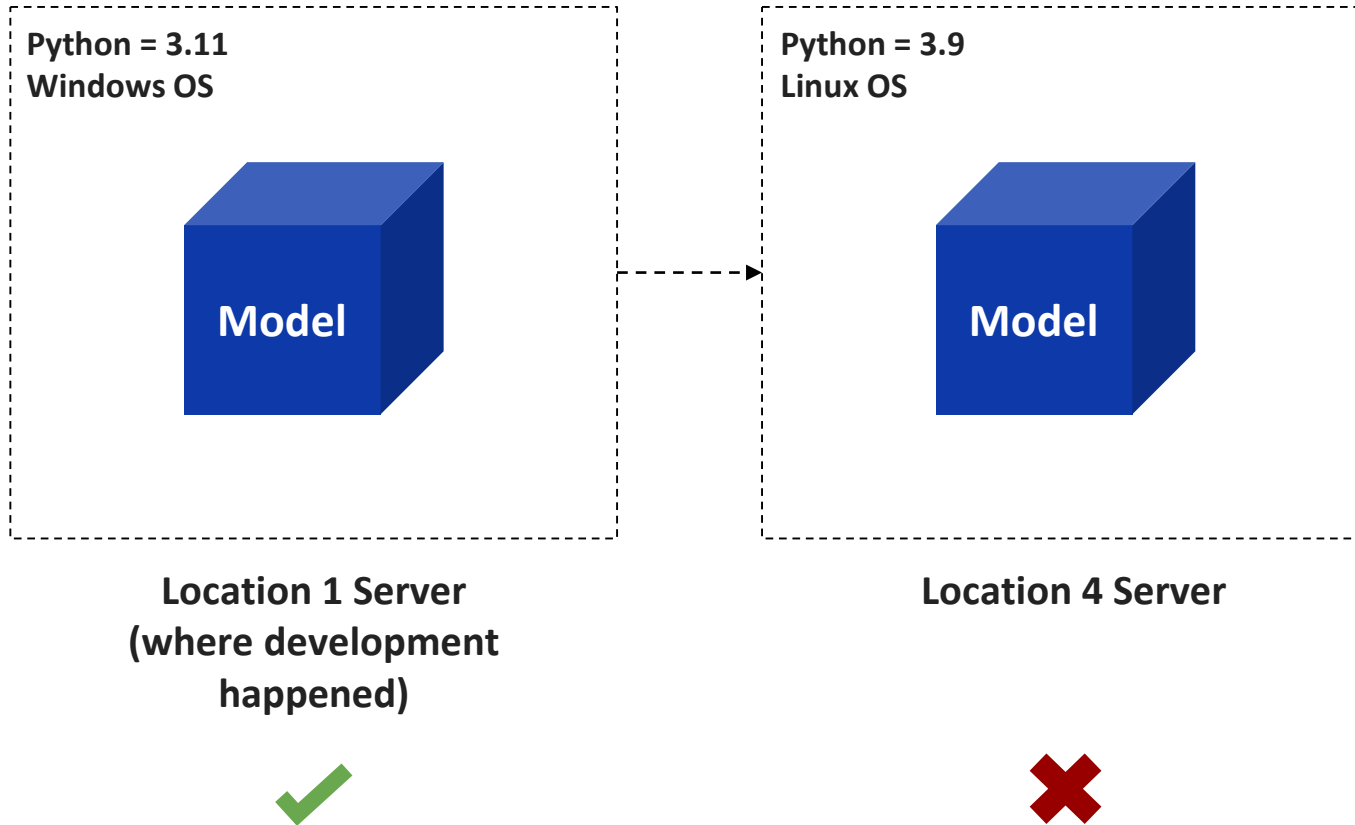


# Challenges with Model Deployment



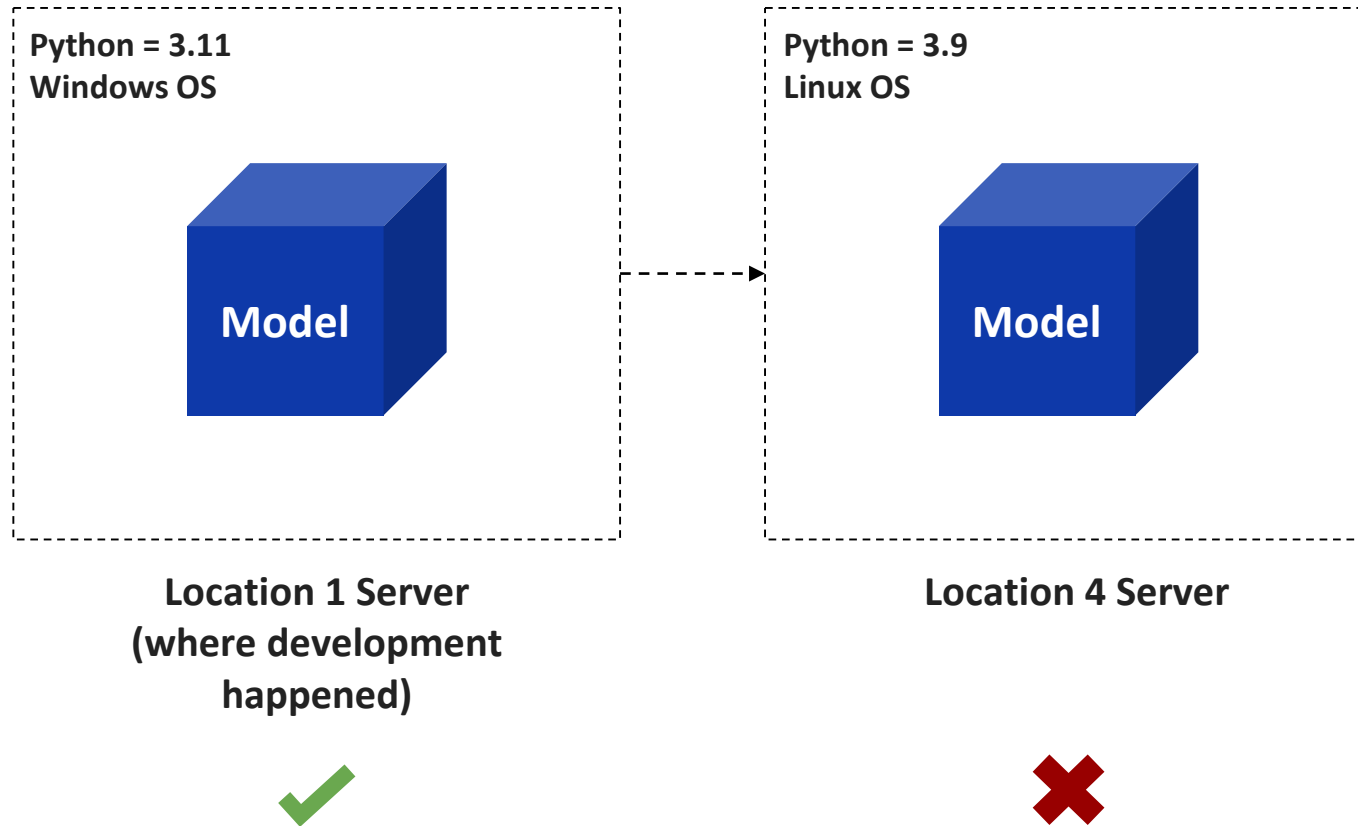
- A **simple approach** to eliminate these issues can be to **share all necessary resources** to servers in **every geographical location**.

# Challenges with Model Deployment



- **Location 1:** Deployment Successful
- **Location 4:** Deployment Failed
- **WHY?**

# Challenges with Model Deployment



- **Location 1:** Deployment Successful
- **Location 4:** Deployment Failed
- **WHY?**

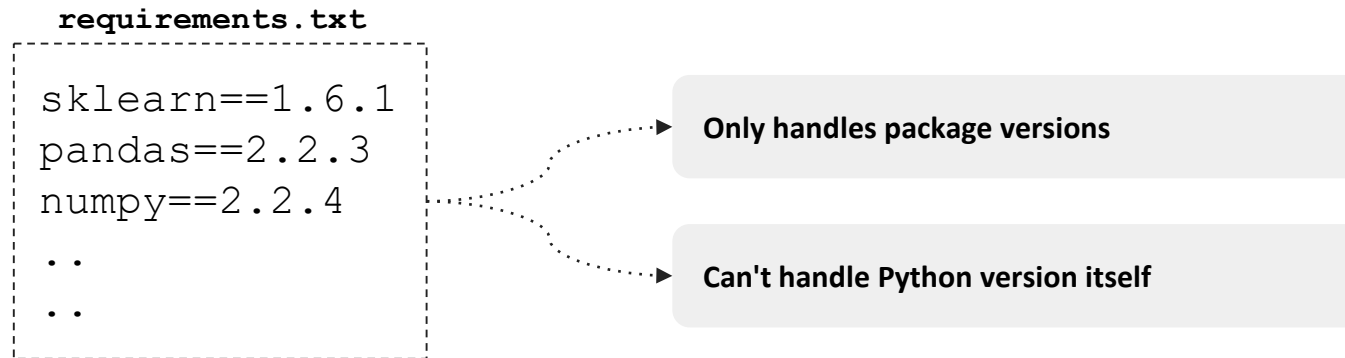
Failure occurred due to **environment** and **Operating System (OS)** configuration inconsistencies.

# Challenges with Model Deployment

- Why can't we just **specify** the **Python version** in dependency file (**requirements.txt**)?

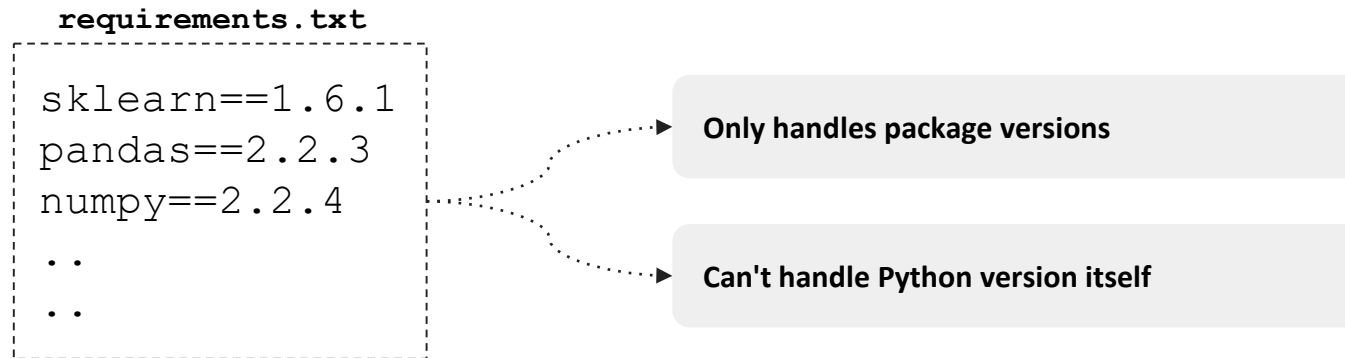
# Challenges with Model Deployment

- Why can't we just **specify** the **Python version** in dependency file (**requirements.txt**)?



# Challenges with Model Deployment

- Why can't we just **specify** the **Python version** in dependency file (**requirements.txt**)?

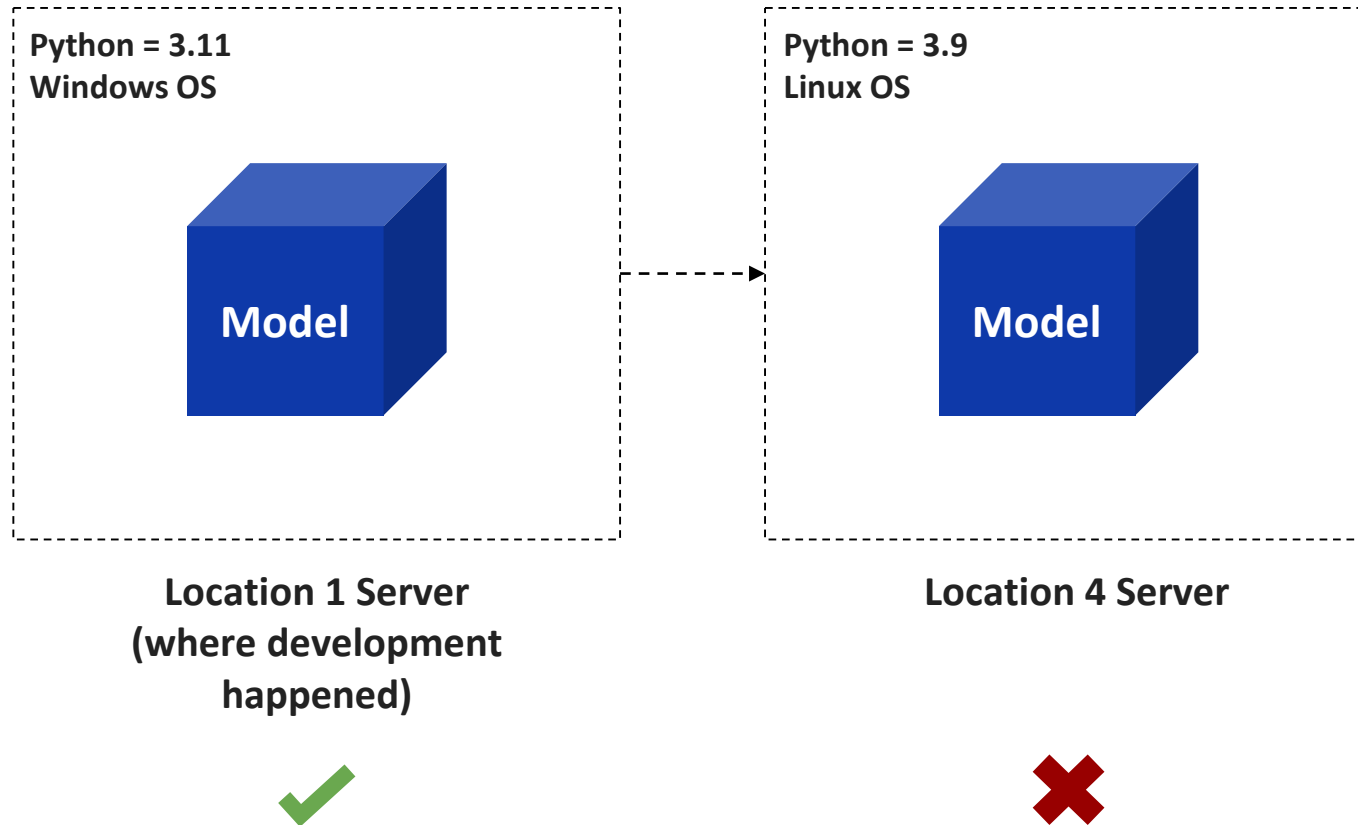


Python libraries **behave differently** on Windows and Linux **due to OS-specific dependencies**.

Package that runs on **Linux might require additional configurations** or even have a **different installation process on Windows**, and vice versa.

# Video Break

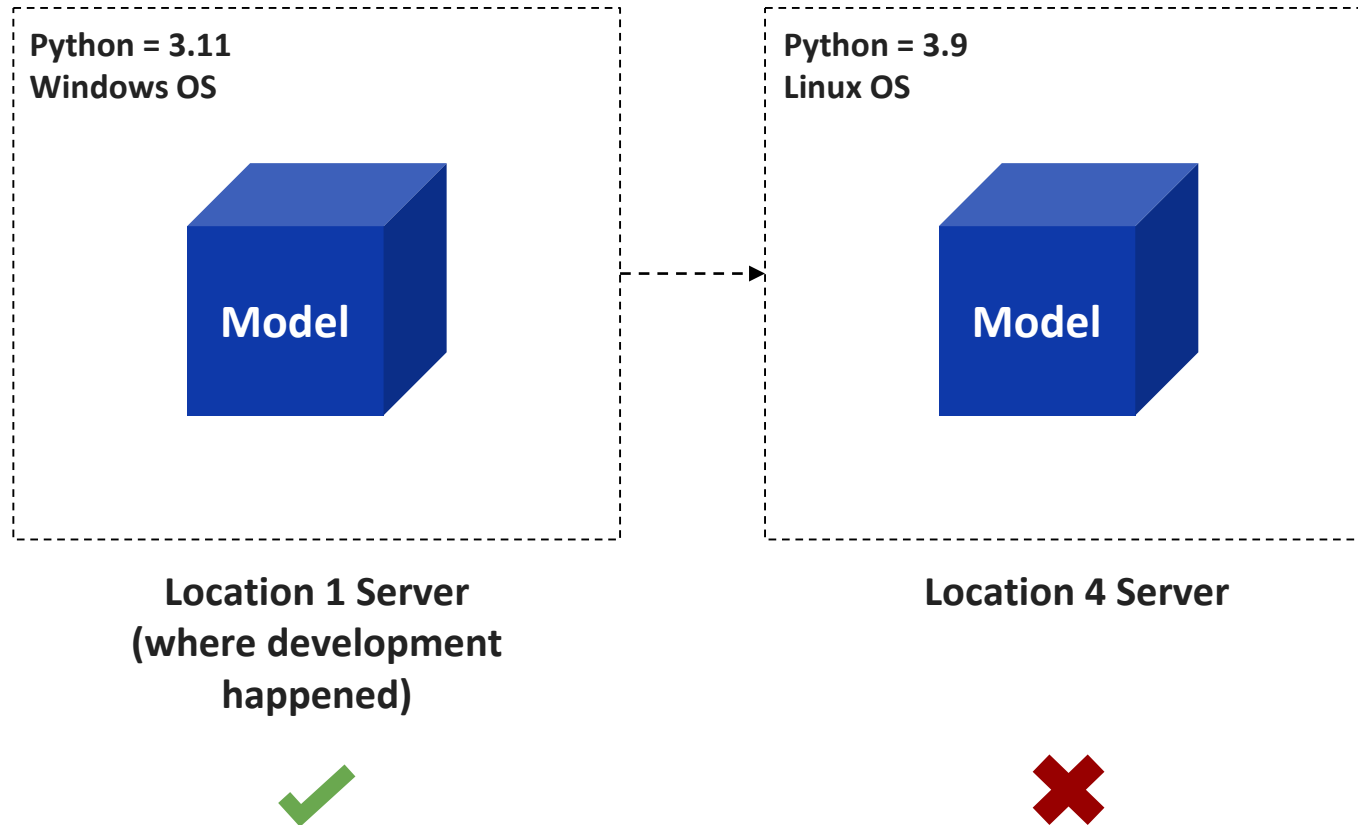
# Need for Virtualization and Containerization



We need a way to **create a controlled and consistent setup** for deploying Machine Learning models.



# Need for Virtualization and Containerization

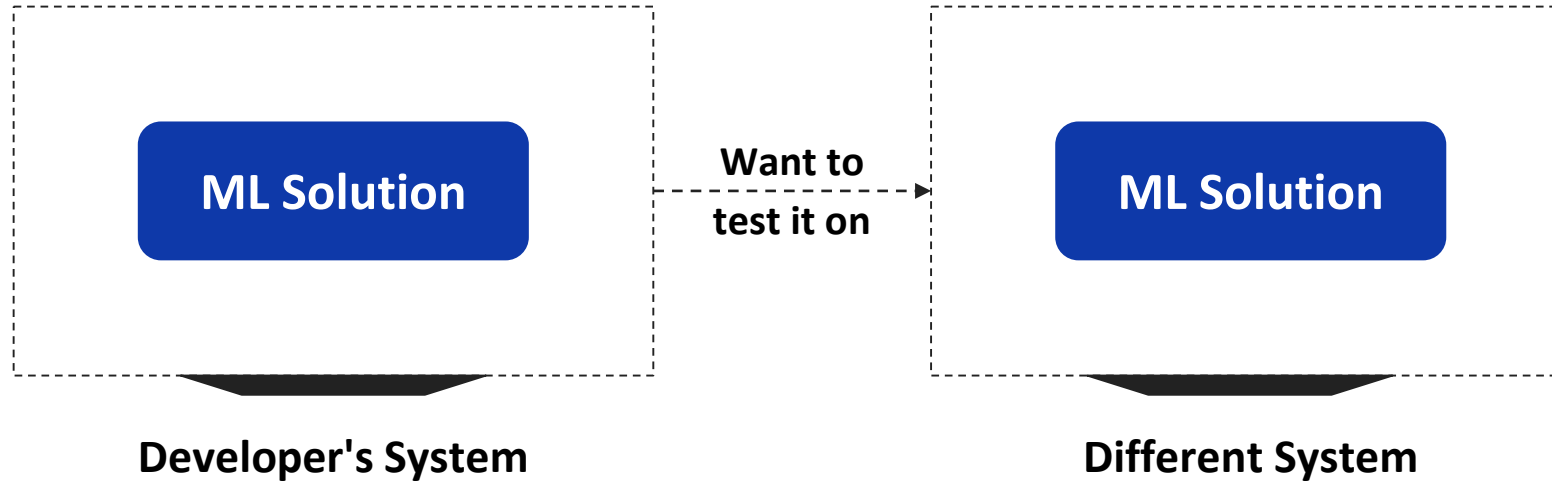


We need a way to **create a controlled and consistent setup** for deploying Machine Learning models.

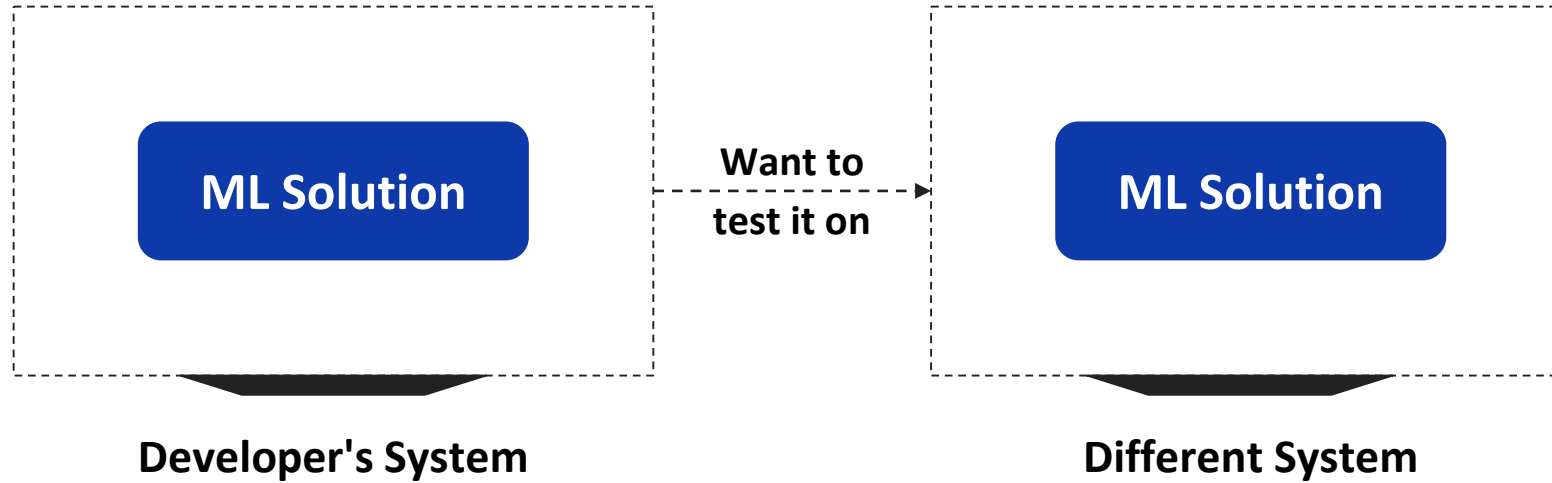
We can then package everything the model needs into the setup so that it runs the same way on any system.

# Need for Virtualization and Containerization

But why?



# Need for Virtualization and Containerization

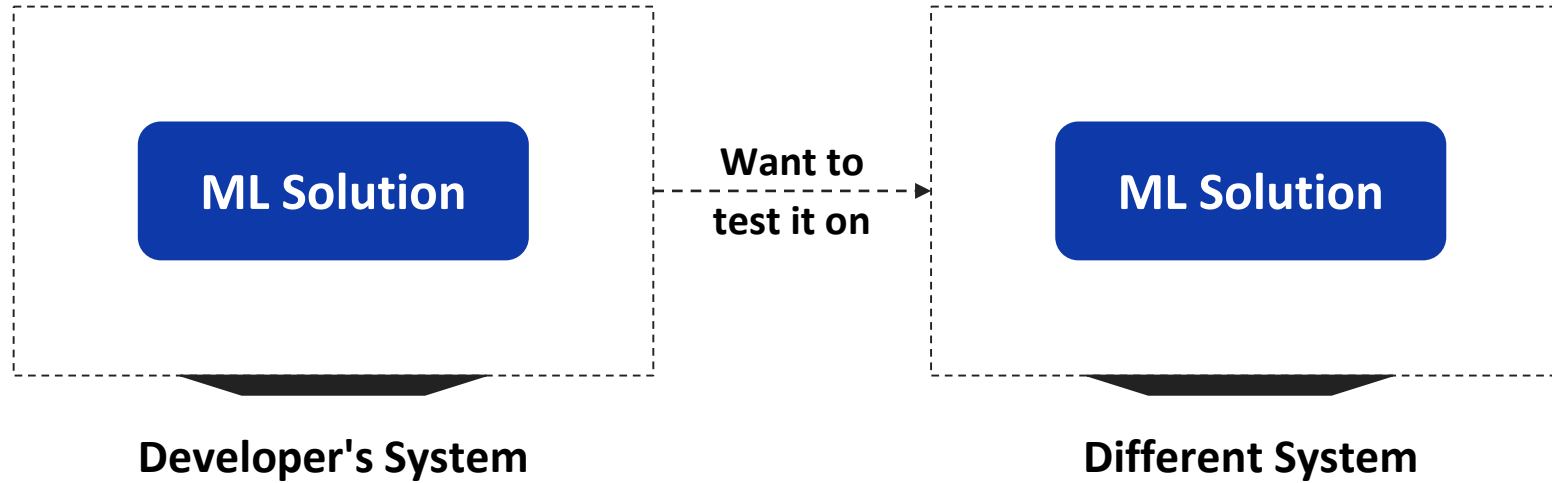


## But why?

To validate that the project **works smoothly** on any system.

To ensure there is **no compatibility** issues.

# Need for Virtualization and Containerization



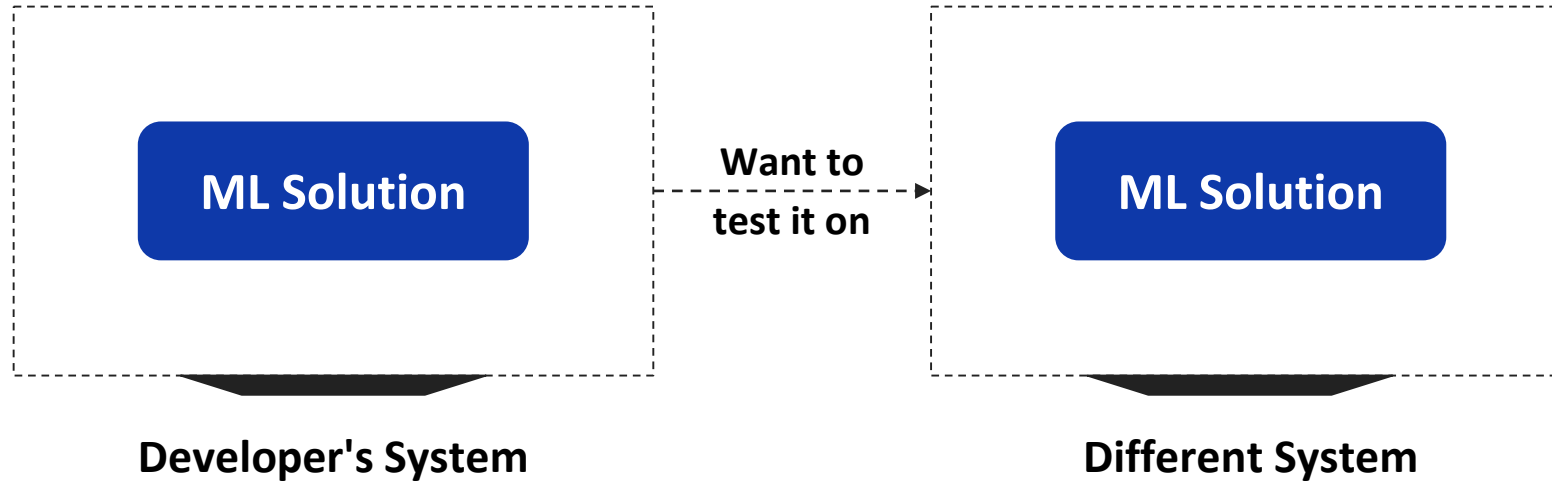
## But why?

To validate that the project **works smoothly** on any system.

To ensure there is **no compatibility** issues.

- Requires us to **buy new hardware** and **setup** to test the ML solution.

# Need for Virtualization and Containerization



## But why?

To validate that the project **works smoothly** on any system.

To ensure there is **no compatibility** issues.

- Requires us to **buy new hardware** and **setup** to test the ML solution.

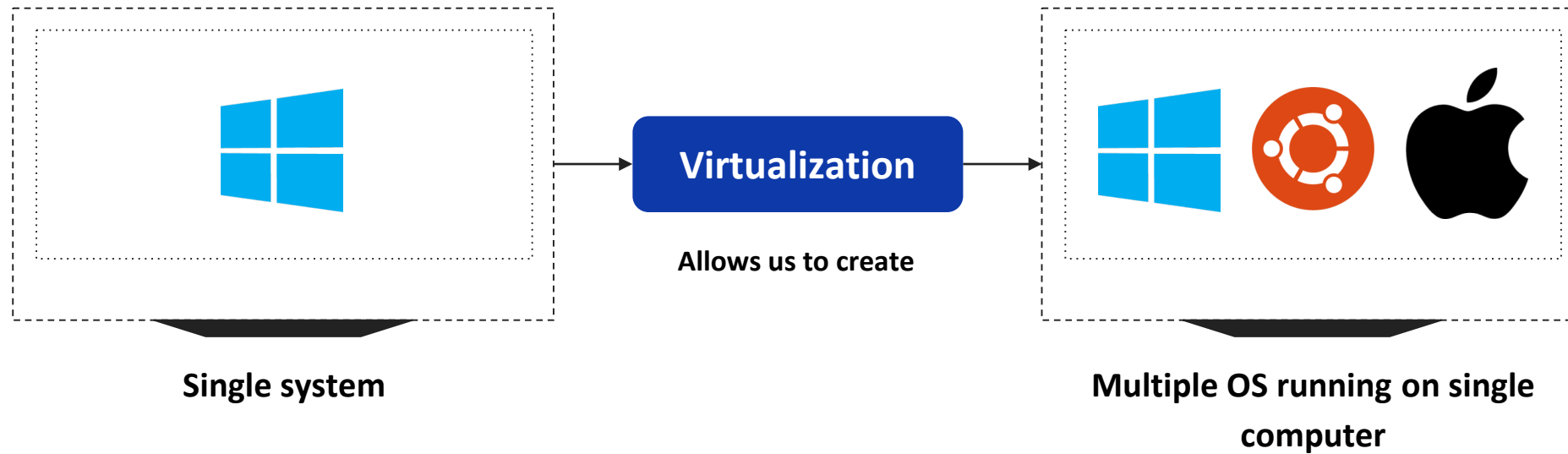
**Virtualization** allows us to do this without actually buying new hardware.

# Virtualization

**Virtualization** is a **technology** that one can use to **create virtual representations** of servers, storage, networks, and other physical machines.

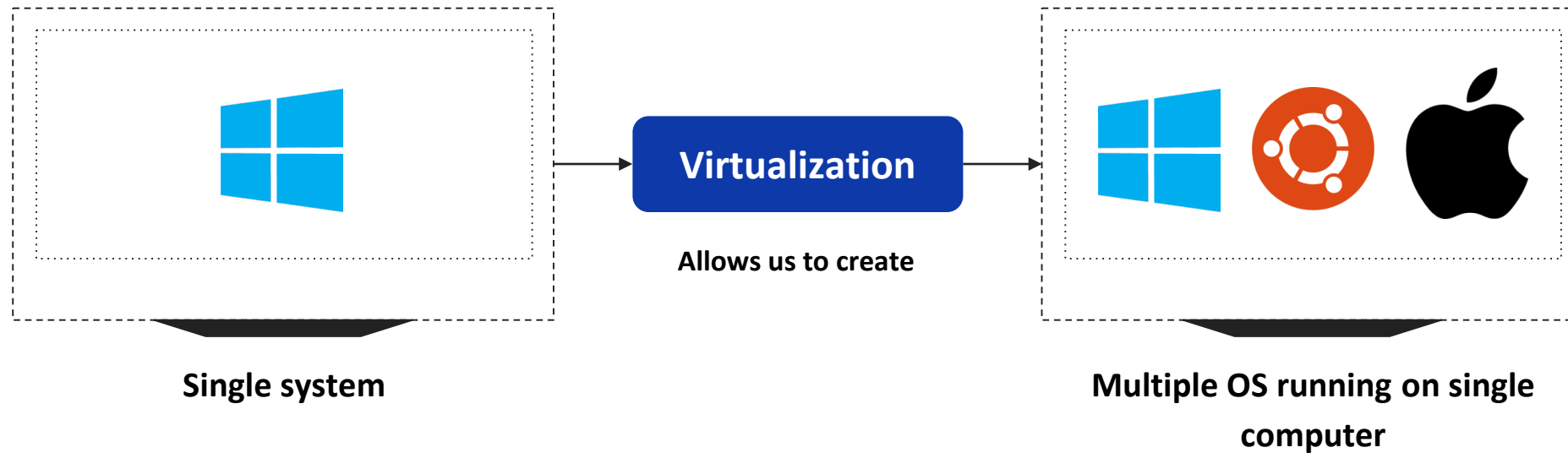
# Virtualization

**Virtualization** is a **technology** that one can use to **create virtual representations** of servers, storage, networks, and other physical machines.



# Virtualization

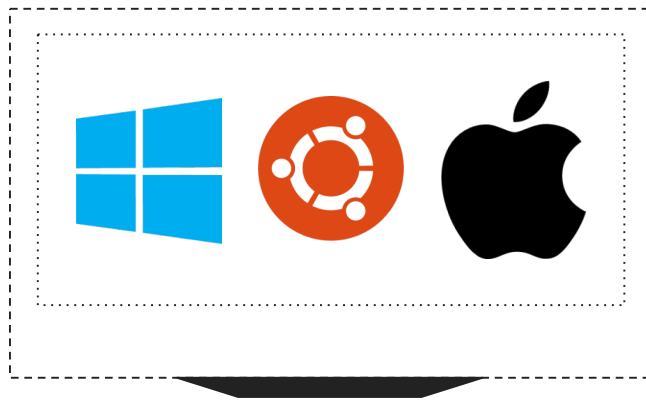
**Virtualization** is a **technology** that one can use to **create virtual representations** of servers, storage, networks, and other physical machines.



- We can create a **clean** and **controlled environment** for the ML model.
- No matter the OS, **virtualization ensures everything runs smoothly** to validate that the **project would work fine on any system**.



# Drawbacks of Virtualization

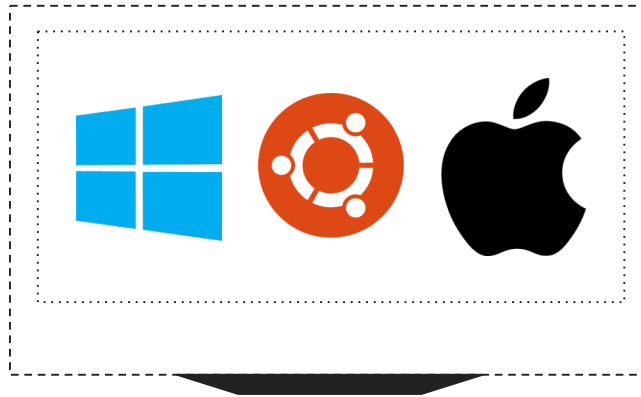


Multiple OS running on single system

Virtualization **requires a lot of memory and processing power** because they run full Operating System(s).

More users increase workload, slowing down inference if resources are not scaled accordingly.

# Drawbacks of Virtualization



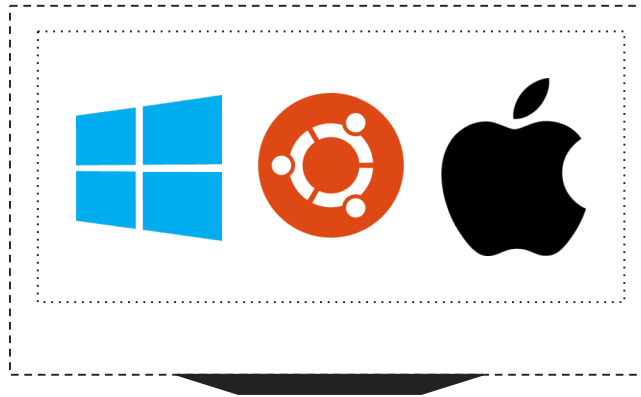
**Multiple OS running on single system**

Virtualization **requires a lot of memory and processing power** because they run full Operating System(s).

More users increase workload, slowing down inference if resources are not scaled accordingly.

Scaling means adding more compute resources, which is generally done by upgrading hardware (RAM, CPU, storage).

# Drawbacks of Virtualization



Multiple OS running on single system

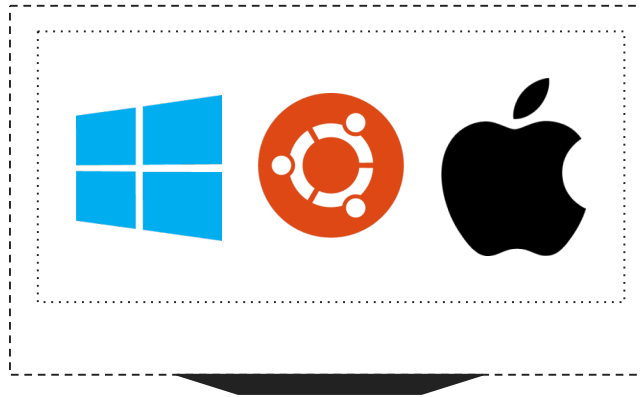
Virtualization **requires a lot of memory and processing power** because they run full Operating System(s).

More users increase workload, slowing down inference if resources are not scaled accordingly.

Scaling means adding more compute resources, which is generally done by upgrading hardware (RAM, CPU, storage).

- What if we could avoid building an extra system and instead just have a **small, portable setup** that includes all the necessary components?

# Drawbacks of Virtualization



Multiple OS running on single system

Virtualization **requires a lot of memory and processing power** because they run full Operating System(s).

More users increase workload, slowing down inference if resources are not scaled accordingly.

Scaling means adding more compute resources, which is generally done by upgrading hardware (RAM, CPU, storage).

- What if we could avoid building an extra system and instead just have a **small, portable setup** that includes all the necessary components?

That's where **containerization** comes in.

# Containerization

**Containerization** is the **packaging of software** code with just the **Operating System (OS) libraries and dependencies** required to run the code, creating a **single lightweight executable** called a **container** that runs **consistently on any infrastructure**.

# Containerization

Containerization is the **packaging of software** code with just the **Operating System (OS) libraries and dependencies** required to run the code, creating a **single lightweight executable** called a **container** that runs **consistently on any infrastructure**.

- No need to install everything manually — **just run the container**.
- It **works in the same way** on every OS, whether it is on Windows, macOS, or Linux.
- Everything is **packaged into a single unit**, making sharing and deploying models effortless.

# Containerization

Containerization is the **packaging of software** code with just the **Operating System (OS) libraries and dependencies** required to run the code, creating a **single lightweight executable** called a **container** that runs **consistently on any infrastructure**.

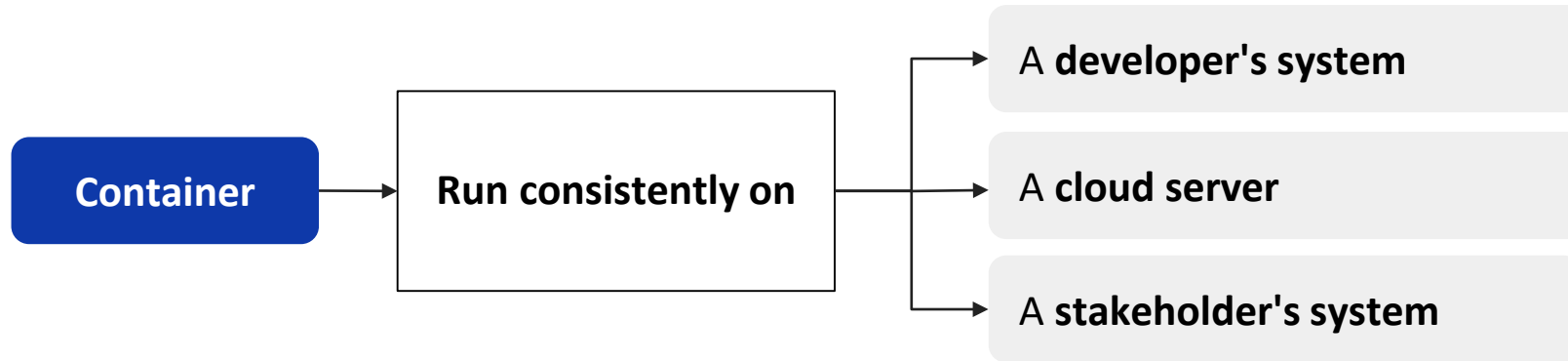
- No need to install everything manually — **just run the container**.
- It **works in the same way** on every OS, whether it is on Windows, macOS, or Linux.
- Everything is **packaged into a single unit**, making sharing and deploying models effortless.

Unlike virtualization, **containers don't need to run full Operating System(s)** as they use only the necessary files to create a consistent environment.

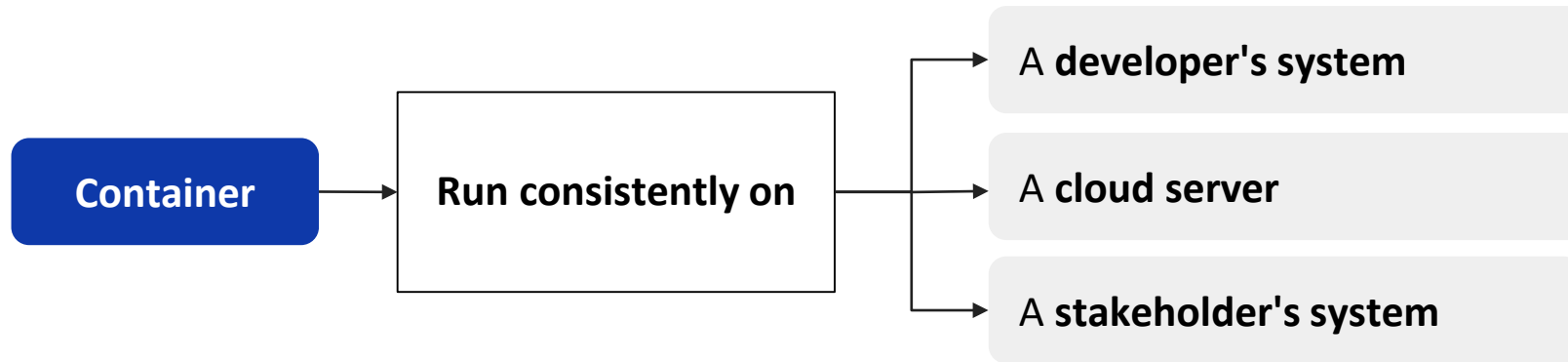
# Video Break



# Need for Base Image

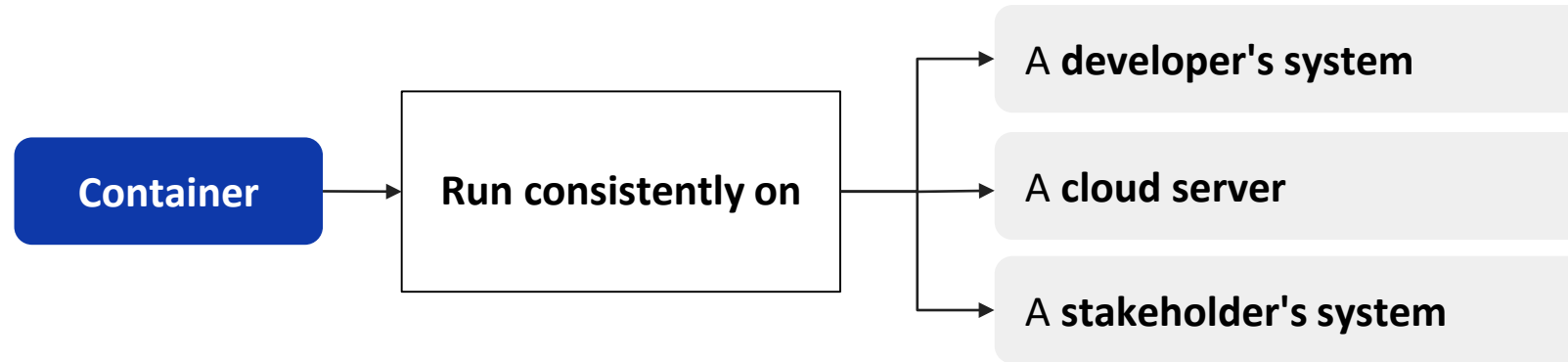


# Need for Base Image



- How do we ensure that **every container** has the **right environment** to **run smoothly**?

# Need for Base Image

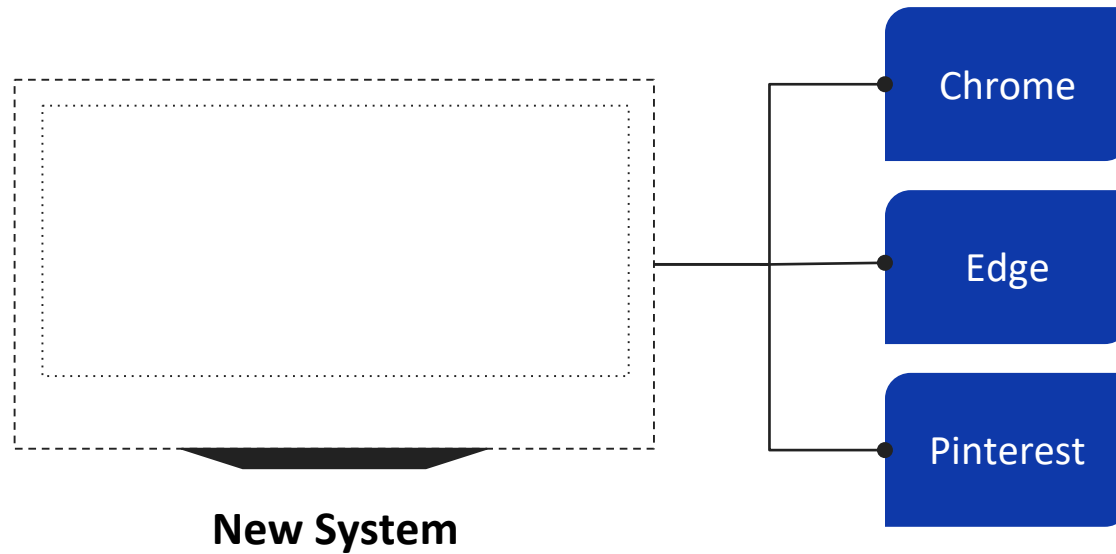


- How do we ensure that **every container** has the **right environment** to **run smoothly**?

While containers solve many problems, they **don't start completely from scratch**.

Each container **needs a base environment** to function.

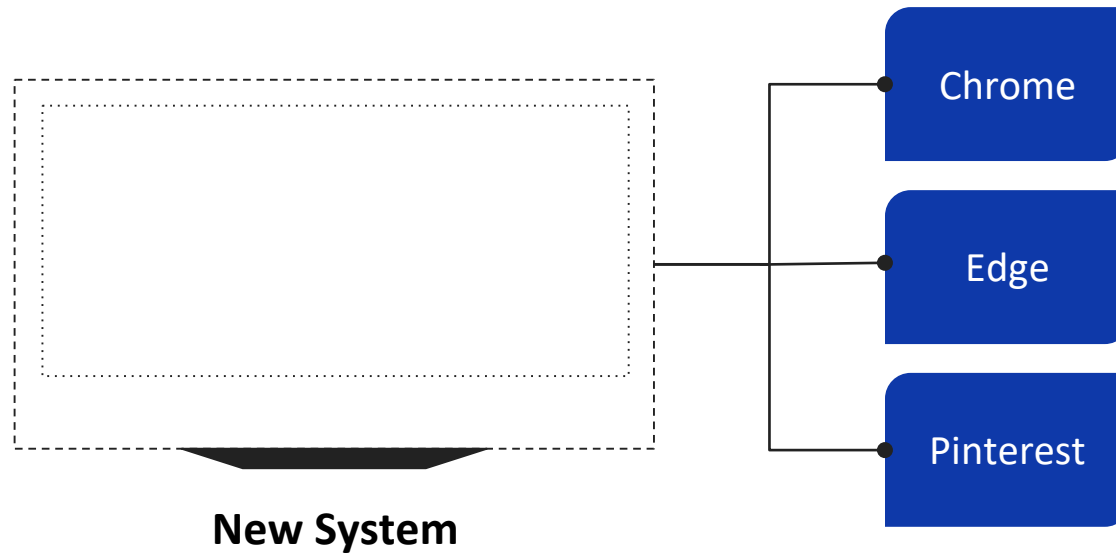
# Need for Base Image



Before we install apps, we need a **basic OS** like Windows, macOS, or Linux.

Without it, **software won't run** the apps.

# Need for Base Image

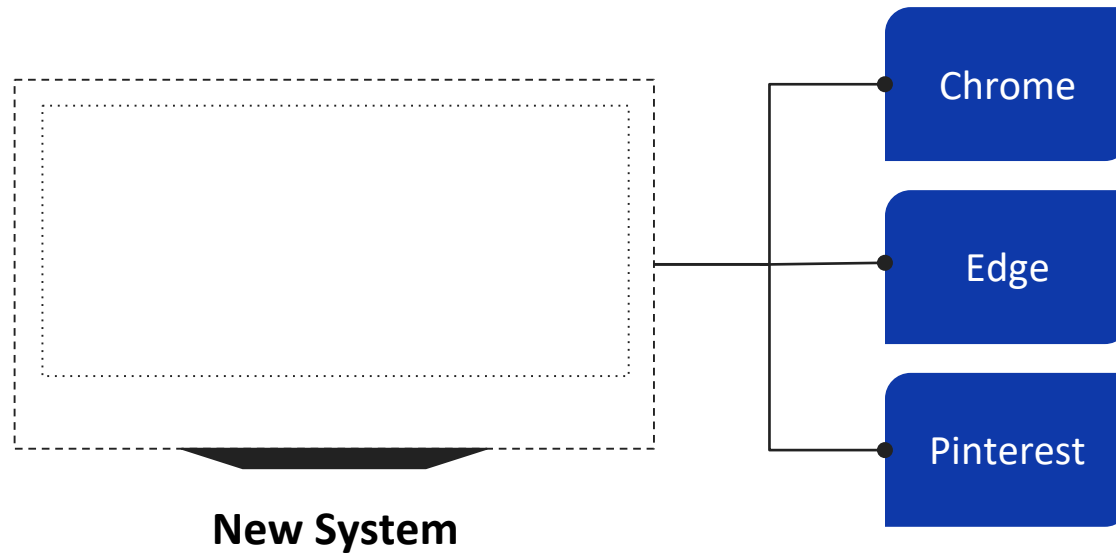


Before we install apps, we need a **basic OS** like Windows, macOS, or Linux.

Without it, **software won't run** the apps.

- Consider deploying an application inside a container.
- If we just add our code without a pre-existing environment, the container won't know which OS or dependencies to use.

# Need for Base Image



Before we install apps, we need a **basic OS** like Windows, macOS, or Linux.

Without it, **software won't run** the apps.

- Consider deploying an application inside a container.
- If we just add our code without a pre-existing environment, the container won't know which OS or dependencies to use.

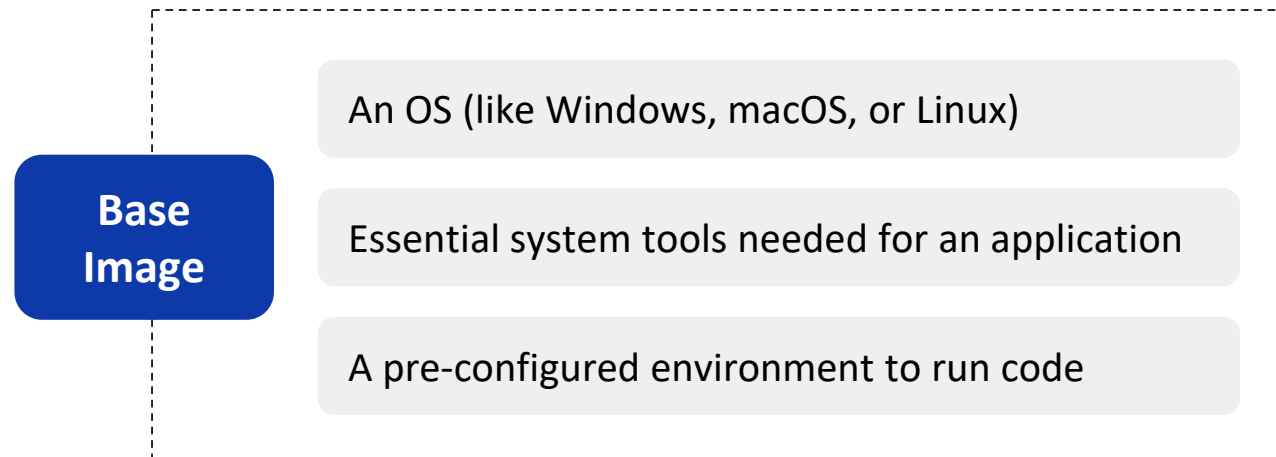
That's why we need a **base image**.

# Base Image

A **base image** is the **starting point** for most container-based development workflows.

# Base Image

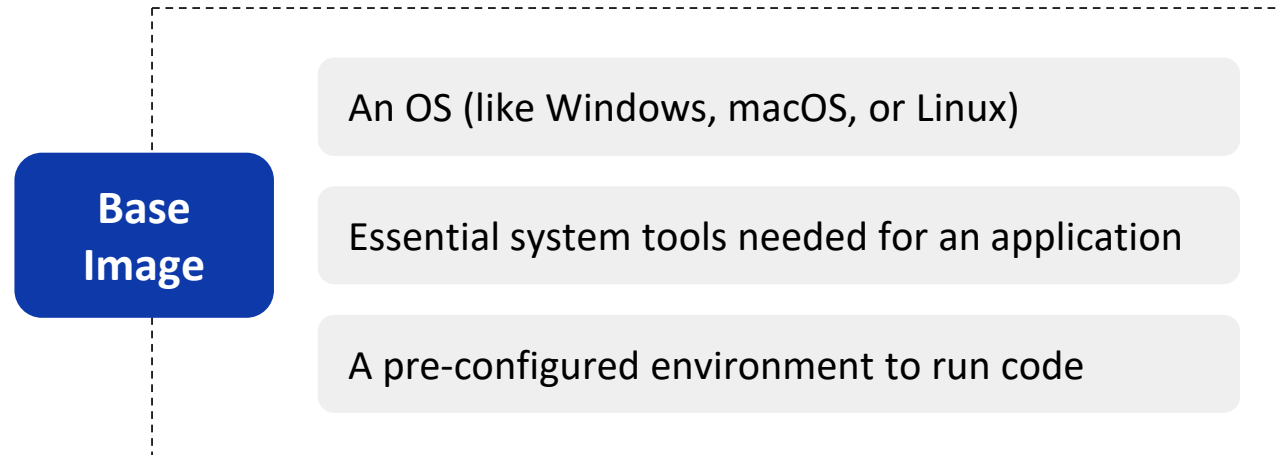
A **base image** is the **starting point** for most container-based development workflows.





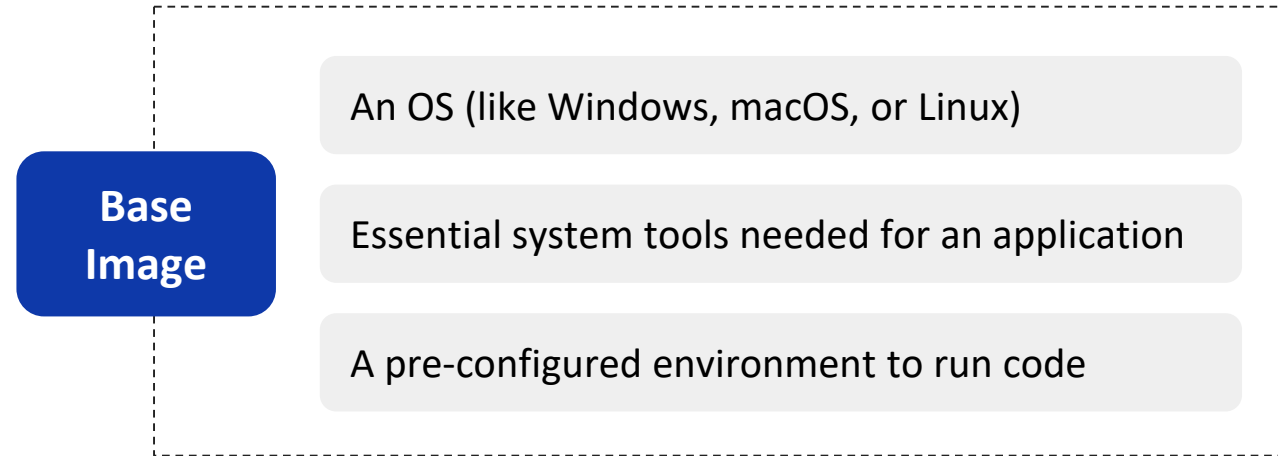
# Base Image

A **base image** is the **starting point** for most container-based development workflows.



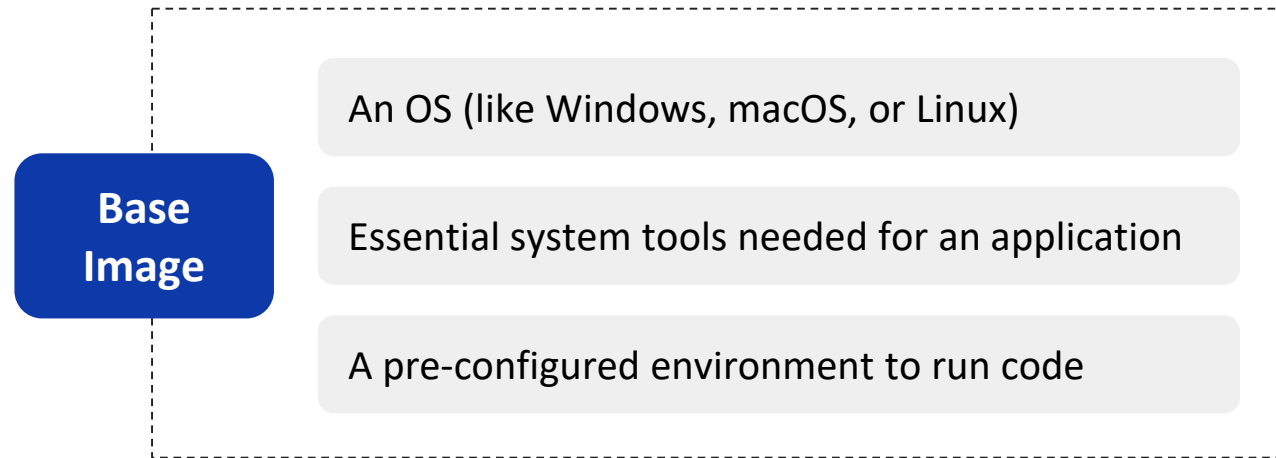
- It provides a **foundational pre-configured environment** for anyone to start building on top of.
- We **start with a base image** and then **add layers** of necessary libraries and configuration files needed for an application **on top of it**.

# Need for Container Image



- But a base image only contains the foundational blocks.
- We'll need the additional layers to build our application (content recommendation system).

# Need for Container Image



- But a base image only contains the foundational blocks.
- We'll need the additional layers to build our application (content recommendation system).

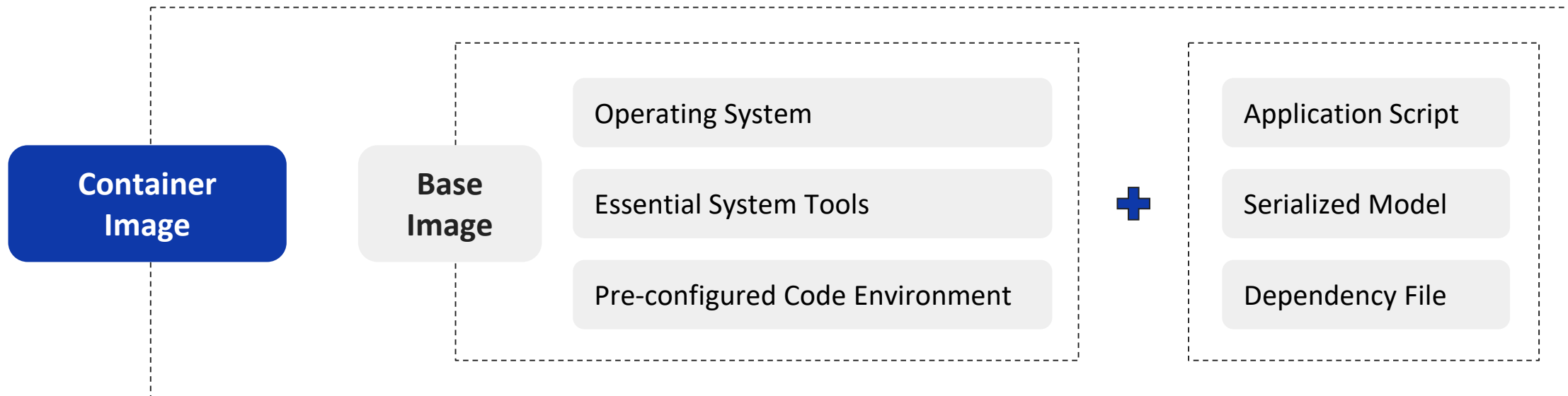
That's where a **container image** comes into the picture.

# Container Image

A **container image** represents binary data that **encapsulates an application and all its software dependencies**.

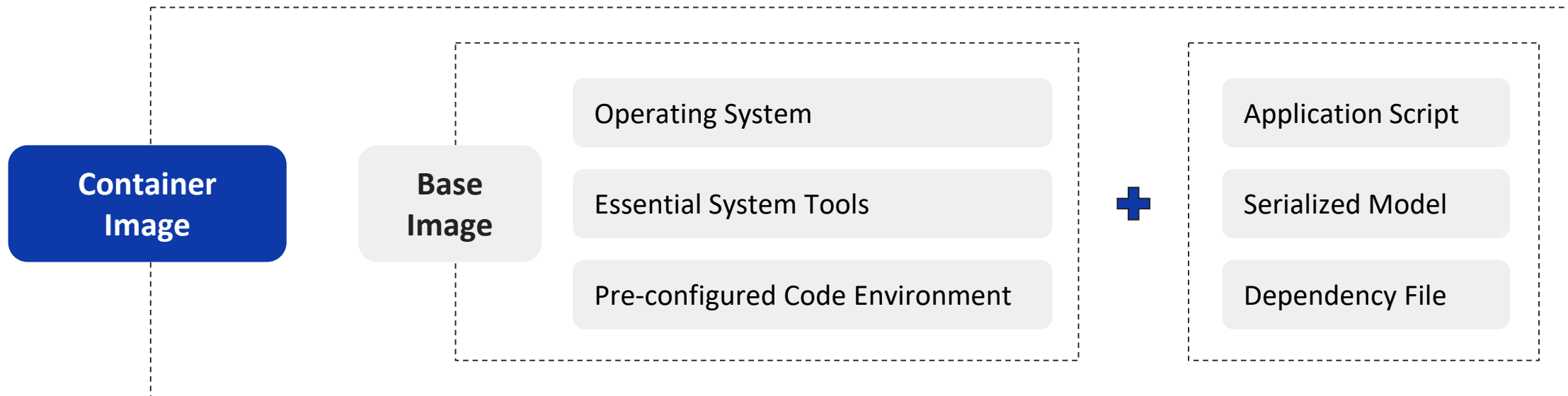
# Container Image

A **container image** represents binary data that **encapsulates an application and all its software dependencies**.



# Container Image

A **container image** represents binary data that **encapsulates an application and all its software dependencies**.



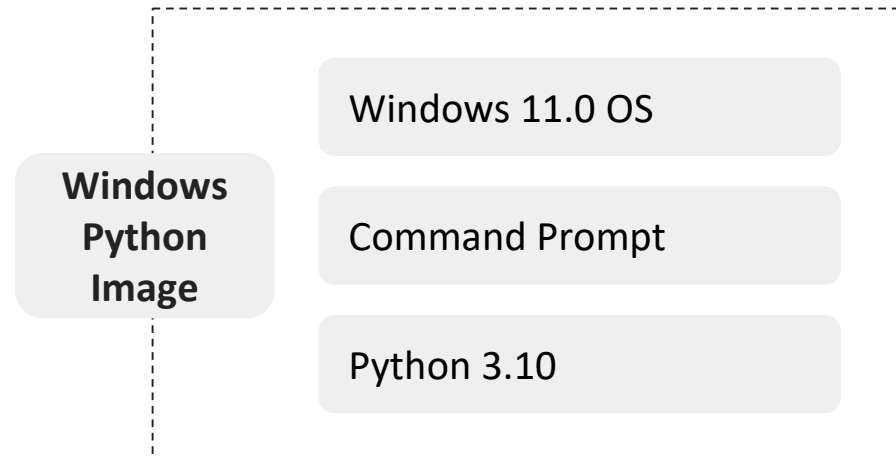
- These are **immutable templates** that **assemble all necessary components** and **can run standalone**.

# Container Image: Example

- Let's see how this structure would look like for our content recommendation system:

# Container Image: Example

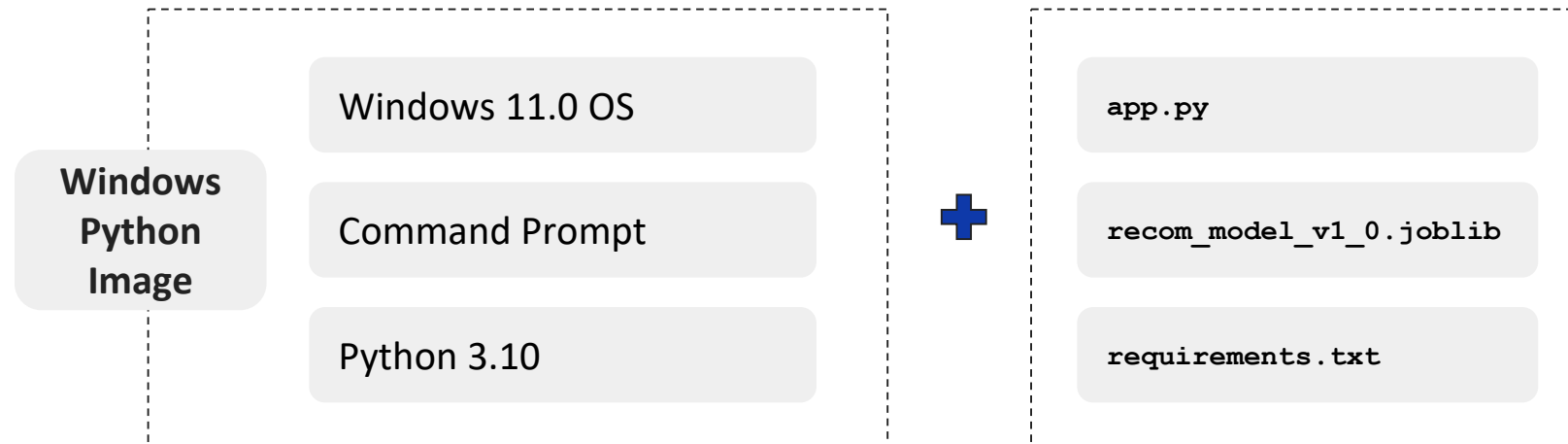
- Let's see how this structure would look like for our content recommendation system:





# Container Image: Example

- Let's see how this structure would look like for our content recommendation system:



# Container Image: Example

- Let's see how this structure would look like for our content recommendation system:



# Video Break

# Need for Dockerfile

- Development teams at different locations were provided with the necessary resources for the content recommendation system.
- They accessed base images as per the system configurations in their locations.

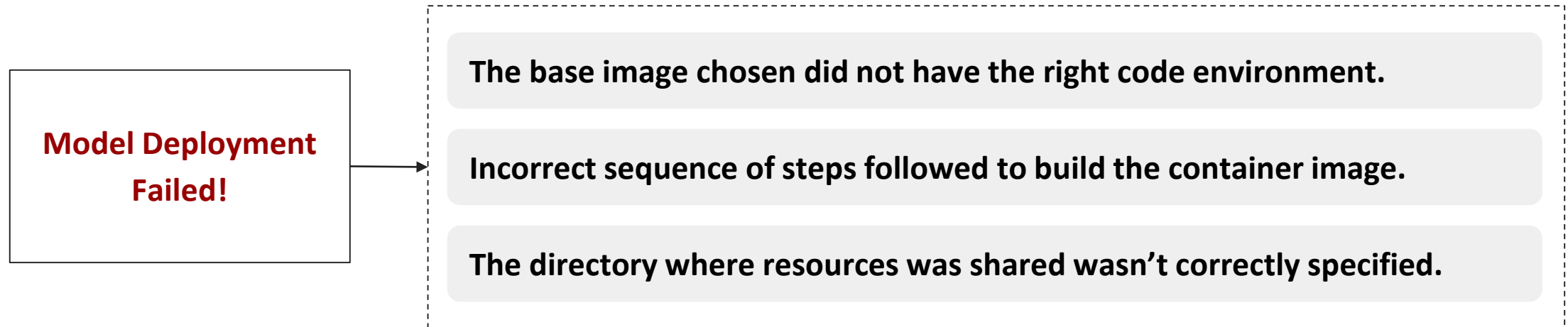
# Need for Dockerfile

- Development teams at different locations were provided with the necessary resources for the content recommendation system.
- They accessed base images as per the system configurations in their locations.

**Model Deployment  
Failed!**

# Need for Dockerfile

- Development teams at different locations were provided with the necessary resources for the content recommendation system.
- They accessed base images as per the system configurations in their locations.



# Dockerfile

**Manually setting up** containers  
is not scalable.

1

# Dockerfile

**Manually setting up** containers  
is **not scalable**.

1

2

**Need a way to automate** how  
each container is built.



# Dockerfile

**Manually setting up** containers is **not scalable**.

1

Dockerfile

2

**Need a way to automate** how each container is built.

A **Dockerfile** is a text document that **contains all the commands** a user requires to assemble a container image.

# Dockerfile



A **Dockerfile** is a text document that **contains all the commands** a user requires to assemble a container image.

- It **provides instructions** on the commands to run, files to copy, when to startup, and more.
- One can **build consistent images automatically** by reading the instructions from a Dockerfile.

# Common Dockerfile Commands

| Command                         | Description          |
|---------------------------------|----------------------|
| <code>FROM &lt;image&gt;</code> | Defines a base image |

# Common Dockerfile Commands

| Command                                | Description                                                                       |
|----------------------------------------|-----------------------------------------------------------------------------------|
| <code>FROM &lt;image&gt;</code>        | Defines a base image                                                              |
| <code>WORKDIR &lt;directory&gt;</code> | Sets the working directory for any instructions that follows it in the Dockerfile |

# Common Dockerfile Commands

| Command                                    | Description                                                                       |
|--------------------------------------------|-----------------------------------------------------------------------------------|
| <code>FROM &lt;image&gt;</code>            | Defines a base image                                                              |
| <code>WORKDIR &lt;directory&gt;</code>     | Sets the working directory for any instructions that follows it in the Dockerfile |
| <code>COPY &lt;src&gt; &lt;dest&gt;</code> | Copies new files or directories and adds them to the file system of the container |

# Common Dockerfile Commands

| Command                                    | Description                                                                       |
|--------------------------------------------|-----------------------------------------------------------------------------------|
| <code>FROM &lt;image&gt;</code>            | Defines a base image                                                              |
| <code>WORKDIR &lt;directory&gt;</code>     | Sets the working directory for any instructions that follows it in the Dockerfile |
| <code>COPY &lt;src&gt; &lt;dest&gt;</code> | Copies new files or directories and adds them to the file system of the container |
| <code>RUN &lt;command&gt;</code>           | Executes any commands in a new layer on top of the base image                     |

# Common Dockerfile Commands

| Command                                    | Description                                                                                       |
|--------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>FROM &lt;image&gt;</code>            | Defines a base image                                                                              |
| <code>WORKDIR &lt;directory&gt;</code>     | Sets the working directory for any instructions that follows it in the Dockerfile                 |
| <code>COPY &lt;src&gt; &lt;dest&gt;</code> | Copies new files or directories and adds them to the file system of the container                 |
| <code>RUN &lt;command&gt;</code>           | Executes any commands in a new layer on top of the base image                                     |
| <code>CMD &lt;command&gt;</code>           | Let's you define the default program that is run once you start the container based on this image |

# Common Dockerfile Commands

| Command                                    | Description                                                                                       |
|--------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>FROM &lt;image&gt;</code>            | Defines a base image                                                                              |
| <code>WORKDIR &lt;directory&gt;</code>     | Sets the working directory for any instructions that follows it in the Dockerfile                 |
| <code>COPY &lt;src&gt; &lt;dest&gt;</code> | Copies new files or directories and adds them to the file system of the container                 |
| <code>RUN &lt;command&gt;</code>           | Executes any commands in a new layer on top of the base image                                     |
| <code>CMD &lt;command&gt;</code>           | Let's you define the default program that is run once you start the container based on this image |

- Each Dockerfile should ONLY have one CMD command.
- In case more than one are added, ONLY the last one is considered.



# Dockerfile: Example

- Let's consider the following Dockerfile:

## Dockerfile

```
FROM python:3.9
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

# Dockerfile: Example

- Let's consider the following Dockerfile:

**Dockerfile**

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

## Use Python 3.9 as the Base Image

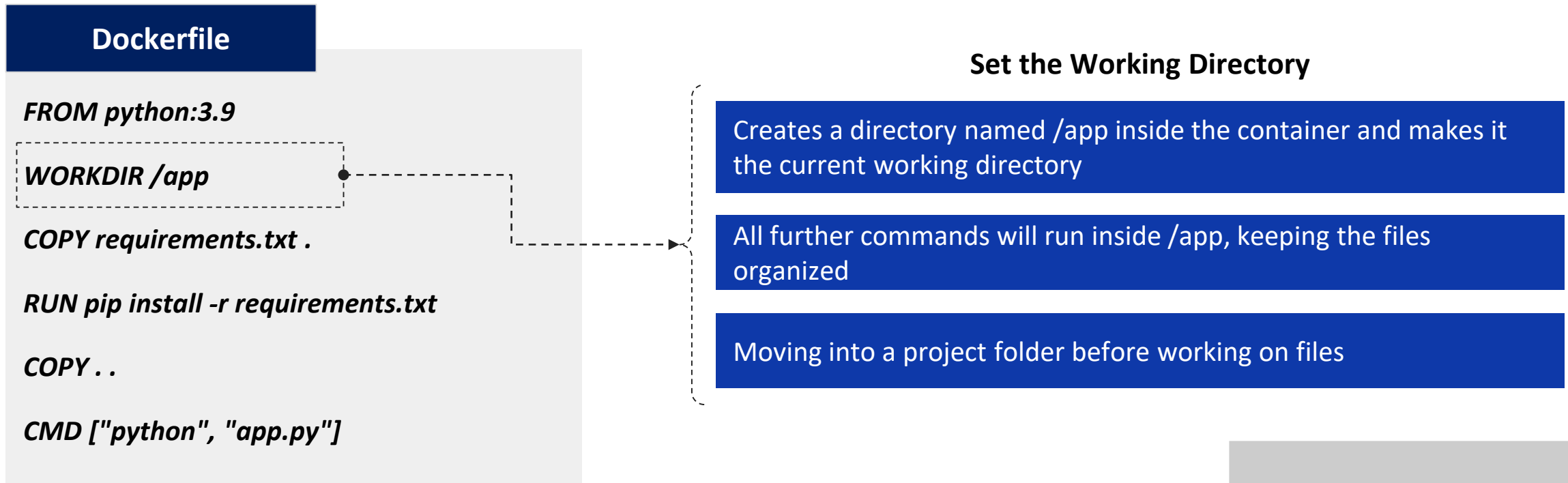
Sets up a base environment with Python 3.9 already installed

Instead of installing Python manually, we use an existing image that includes Python

Choosing a pre-installed laptop instead of setting one up from scratch

# Dockerfile: Example

- Let's consider the following Dockerfile:



# Dockerfile: Example

- Let's consider the following Dockerfile:

## Dockerfile

```
FROM python:3.9
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

## Copy Dependencies

Copies requirements.txt (list of dependencies) from your computer to the container

# Dockerfile: Example

- Let's consider the following Dockerfile:

## Dockerfile

```
FROM python:3.9
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

## Install Libraries

Installs all the Python libraries listed in requirements.txt

Ensures all dependencies are installed in the container

# Dockerfile: Example

- Let's consider the following Dockerfile:

## Dockerfile

```
FROM python:3.9
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "app.py"]
```

## Copy Applications Files

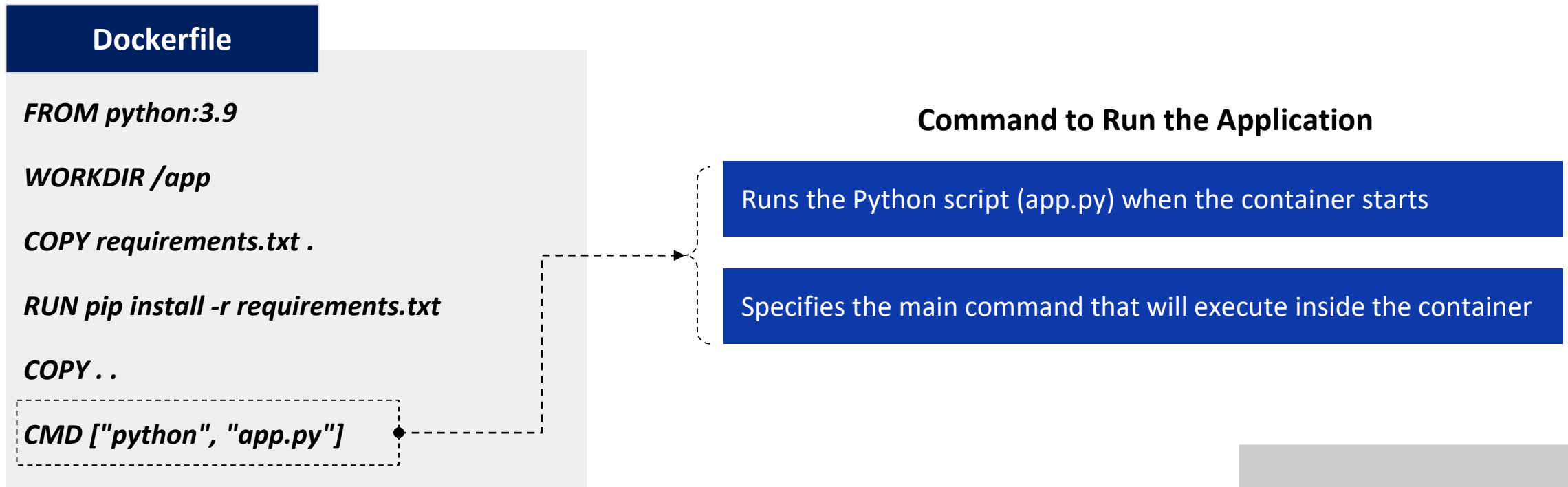
Copies all files from your local project folder to the /app folder in the container

The container needs access to the application files to run the program

Uploading all project files to a server before running a web app

# Dockerfile: Example

- Let's consider the following Dockerfile:



# Video Break



# Need for Container Management

- Containerization is a great approach for packaging an application and all its dependencies together.
- It also allows us to run applications consistently across different environments.

# Need for Container Management

- Containerization is a great approach for packaging an application and all its dependencies together.
- It also allows us to run applications consistently across different environments.

## Challenges

- Manually creating containers can be time-consuming and error-prone.

- Manually managing containers can be complex.

# Need for Container Management

- Containerization is a great approach for packaging an application and all its dependencies together.
- It also allows us to run applications consistently across different environments.

## Challenges

- Manually creating containers can be time-consuming and error-prone.

- Manually managing containers can be complex.

We need a **platform** that can help us overcome these challenges.

# Container Management Platforms

## Docker

One of the most widely used containerization platforms, known for its **ease of use**, **portability**, and **strong community support**.

# Container Management Platforms

## Docker

One of the most widely used containerization platforms, known for its **ease of use, portability, and strong community support.**

## Podman

Similar to Docker, but focuses on **rootless container management.**

# Container Management Platforms

## Docker

One of the most widely used containerization platforms, known for its **ease of use, portability, and strong community support.**

## Podman

Similar to Docker, but focuses on **rootless container management.**

## Openshift

Simplifies the **development, deployment, and management of containerized applications.**

# Container Management Platforms

## Docker

One of the most widely used containerization platforms, known for its **ease of use, portability, and strong community support.**

## Podman

Similar to Docker, but focuses on **rootless container management.**

## Openshift

Simplifies the **development, deployment, and management of containerized applications.**

Among all the available platforms, **Docker stands out as the most beginner-friendly, widely adopted, and versatile solution.**

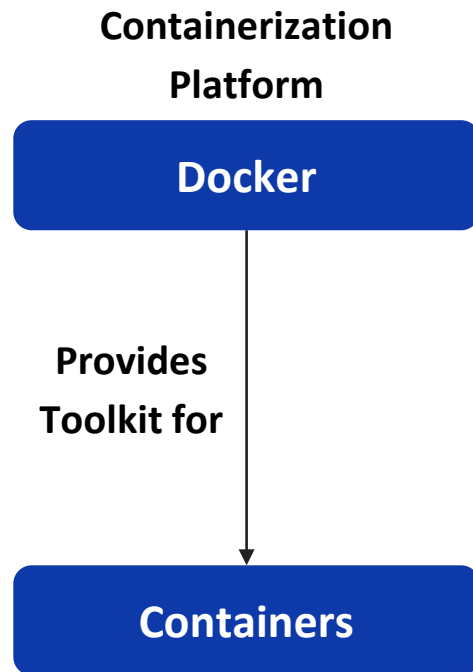
# Docker

**Containerization  
Platform**

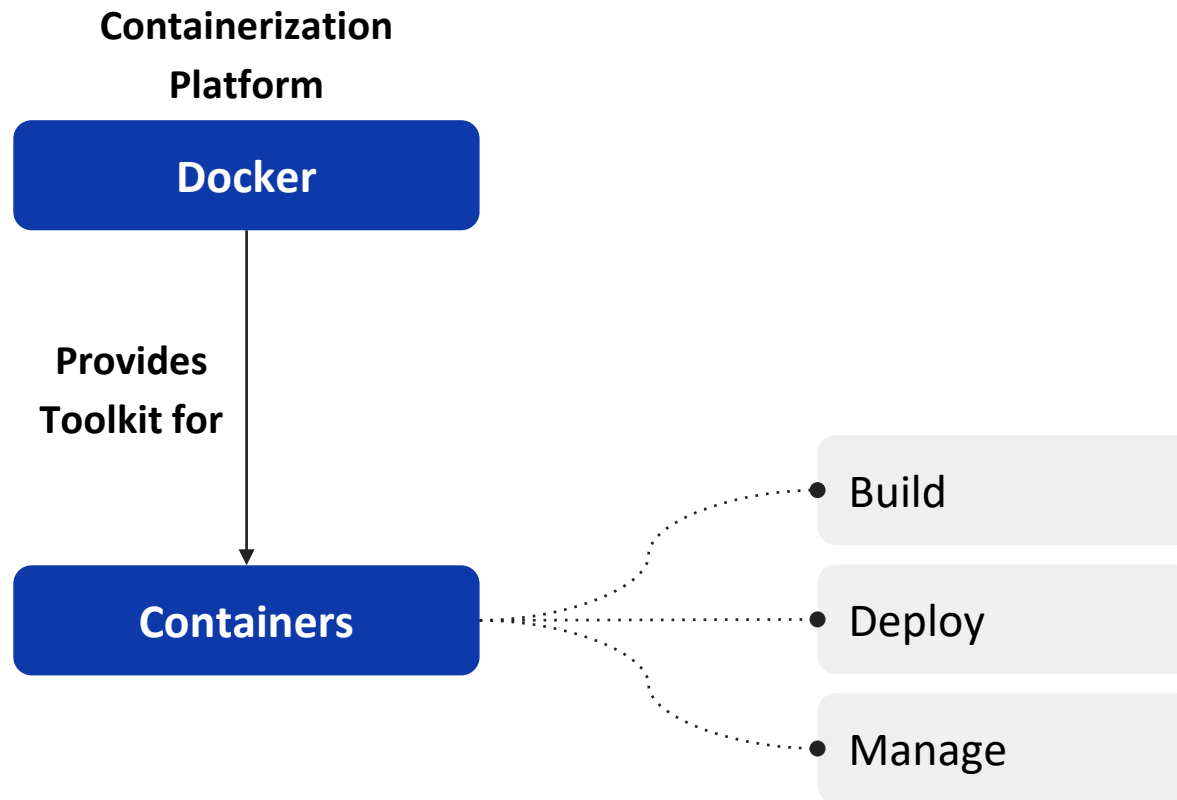
**Docker**



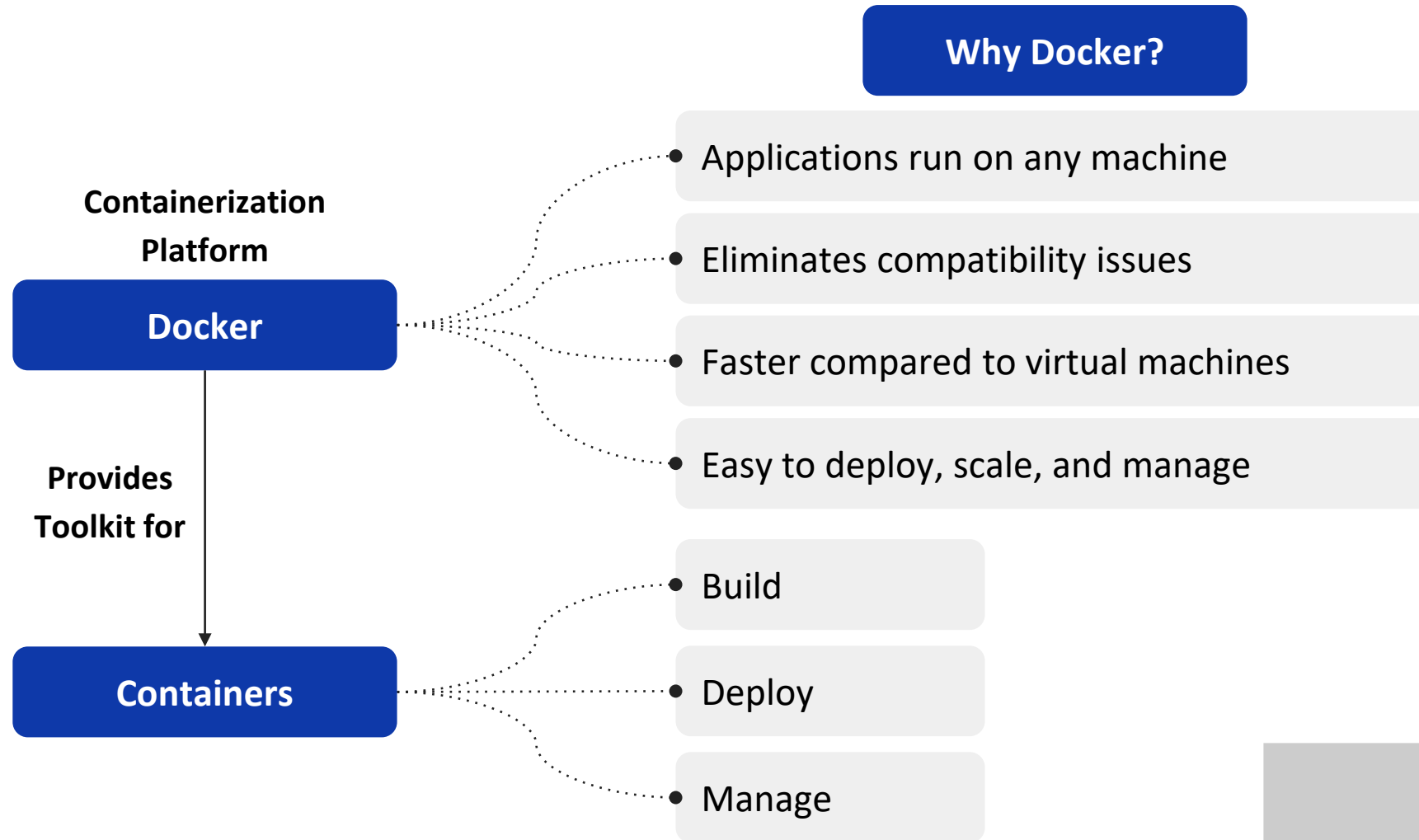
# Docker



# Docker



# Docker

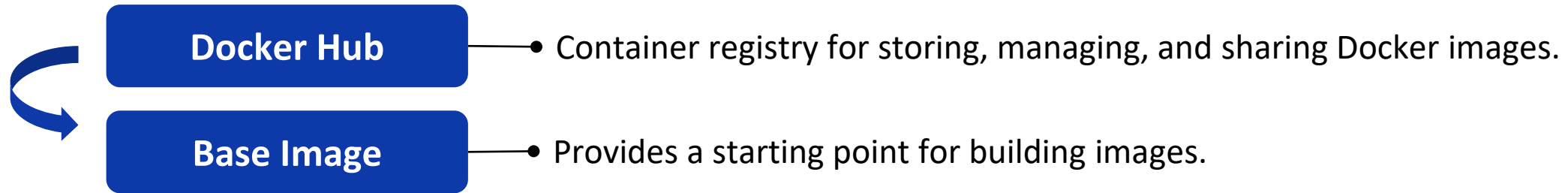


# Containerization with Docker

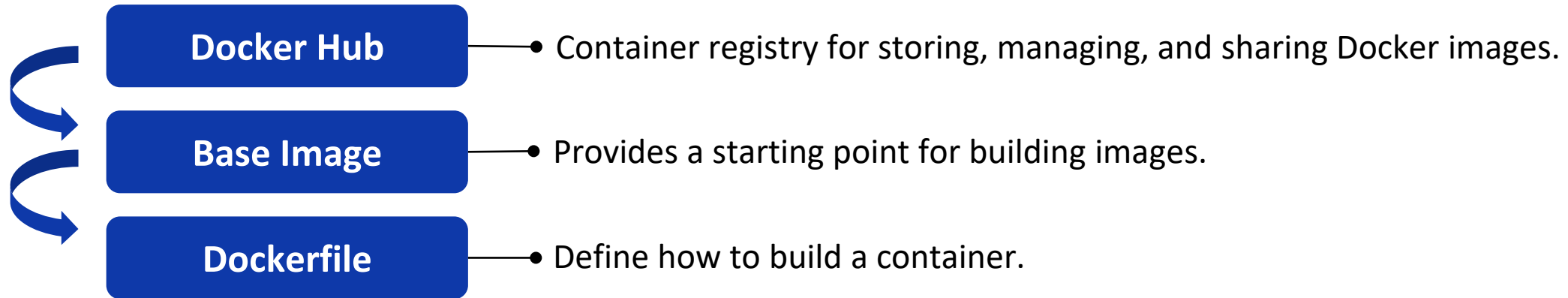
## Docker Hub

- Container registry for storing, managing, and sharing Docker images.

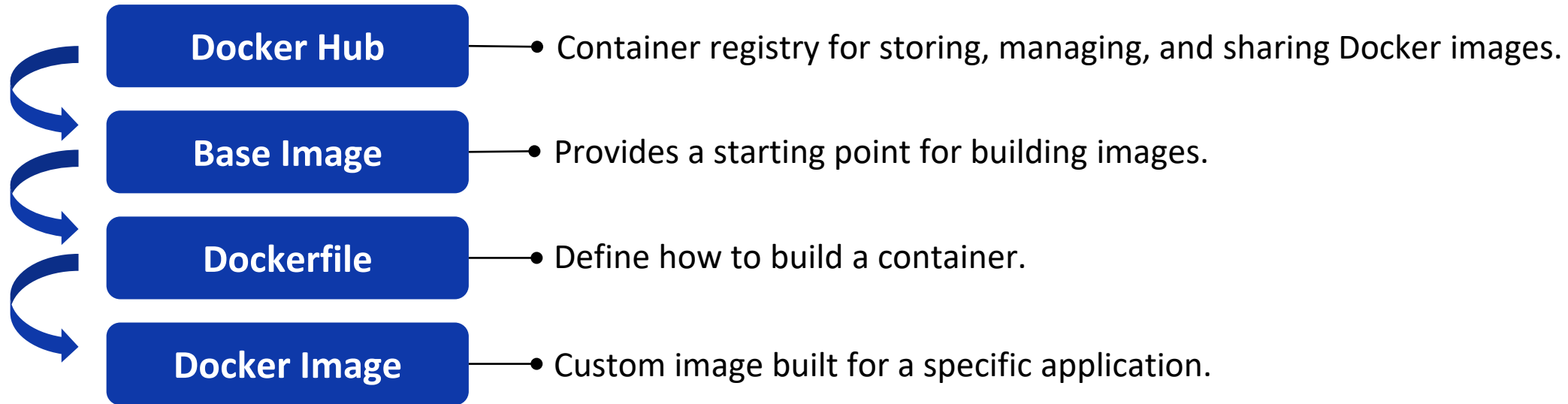
# Containerization with Docker



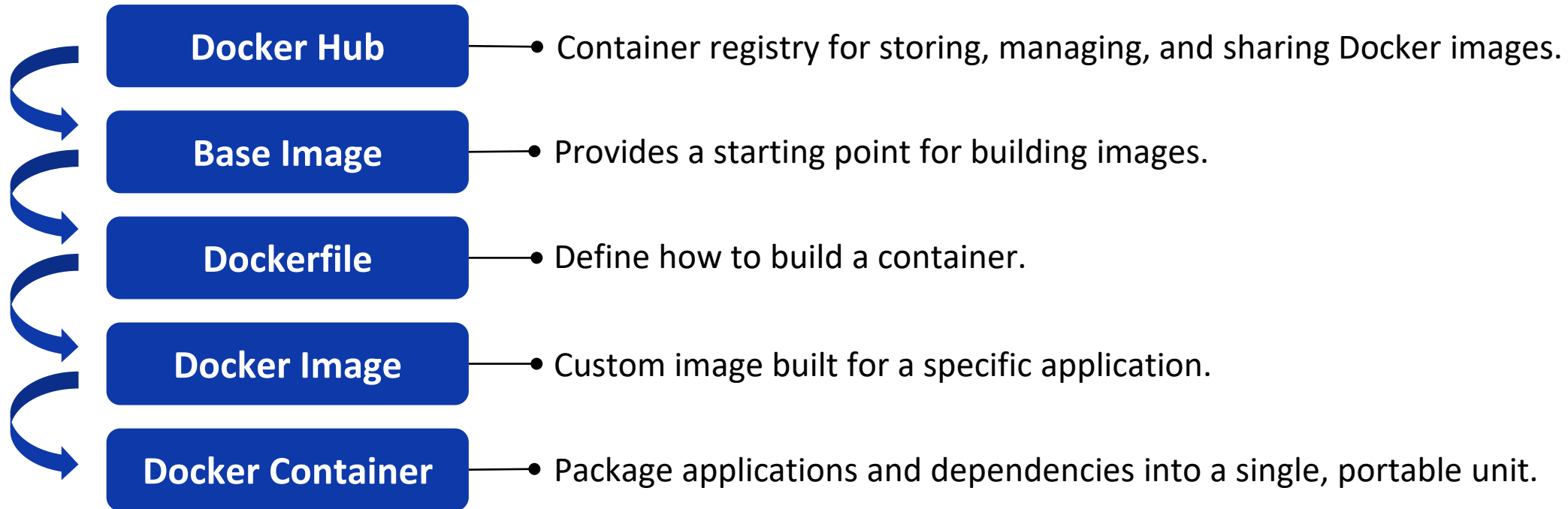
# Containerization with Docker



# Containerization with Docker



# Containerization with Docker

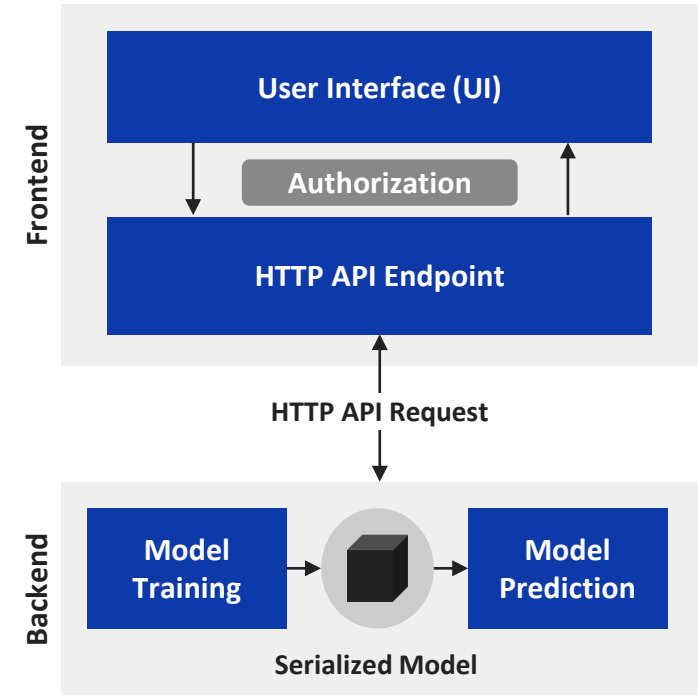




# Video Break

# Need for REST API

- Common model deployment frameworks allow us to build interactive web applications — frontend plus backend.
- If the frontend and backend are coupled together, then developers can't access model outputs.



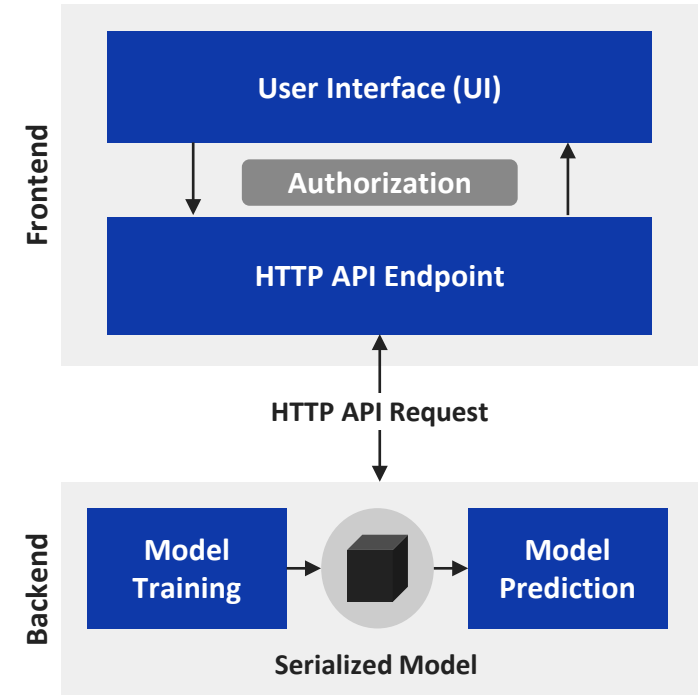
# Need for REST API

- Common model deployment frameworks allow us to build interactive web applications — frontend plus backend.
- If the frontend and backend are coupled together, then developers can't access model outputs.

## Challenges

Changes in the model or its deployment may require significant updates to the web application.

May limit the application's potential by not allowing for easy integration with other services or systems.



# Need for REST API

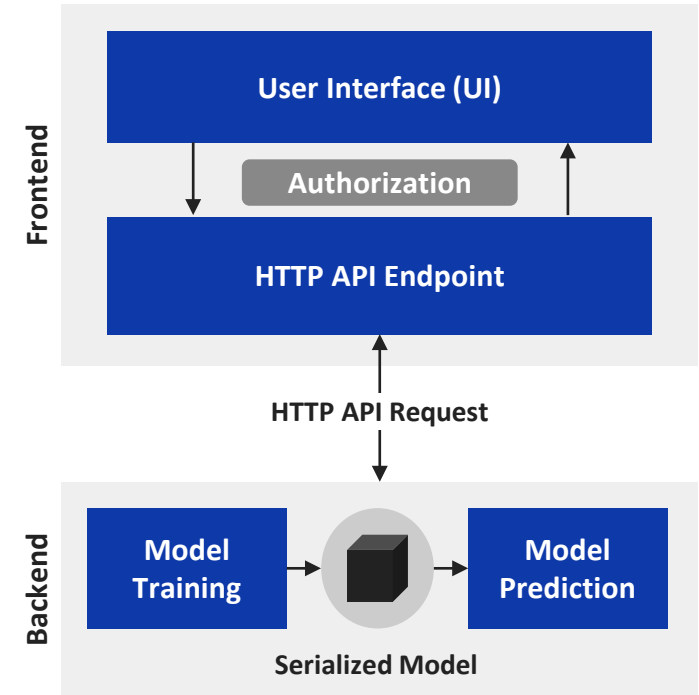
- Common model deployment frameworks allow us to build interactive web applications — frontend plus backend.
- If the frontend and backend are coupled together, then developers can't access model outputs.

## Challenges

Changes in the model or its deployment may require significant updates to the web application.

May limit the application's potential by not allowing for easy integration with other services or systems.

- Accessing the APIs will allow us to fetch both the user input and model output.



# REST API

- A structured, fast, and scalable method of communication is needed.
- Additionally, a **frontend and backend decoupling** is desired, as it will allow for more flexibility.

# REST API

- A structured, fast, and scalable method of communication is needed.
- Additionally, a **frontend and backend decoupling** is desired, as it will allow for more flexibility.

**REST API**, or **RESTful API**, is a **simple, uniform interface** that is used to make data, content, algorithms, media, and other digital resources available through web URLs.

# REST API

- A structured, fast, and scalable method of communication is needed.
- Additionally, a **frontend and backend decoupling** is desired, as it will allow for more flexibility.

**REST API**, or **RESTful API**, is a **simple, uniform interface** that is used to make data, content, algorithms, media, and other digital resources available through web URLs.

It involves a specific set of rules – **Representational State Transfer (REST)** architecture.

**Universal rules** make it simpler for developers to integrate necessary software.

**Decoupling** enables frontend and backend to be developed and enhanced independently.

# API Payload

- APIs need a **structured way to transmitted data** between a client (like a web application) and a server.



# API Payload

- APIs need a **structured way to transmitted data** between a client (like a web application) and a server.

The **payload of an API** refers to the **data** that is sent to or received in a response **when an API request is made**.

# API Payload

- APIs need a **structured way to transmitted data** between a client (like a web application) and a server.

The **payload of an API** refers to the **data** that is sent to or received in a response **when an API request is made**.

- One of the most common formats for API payloads is **JSON** (JavaScript **O**bject **N**otation):

```
{  
  "key": <value(s)>,  
  "key": <value(s)>,  
  "key": <value(s)>,  
}
```

- It is lightweight, widely-used, and language-agnostic.

# Payload: Example

- Let's consider an example of input and output payload for a content recommendation model:

**Recommendation  
Model**



# Payload: Example

- Let's consider an example of input and output payload for a content recommendation model:

## Input Payload

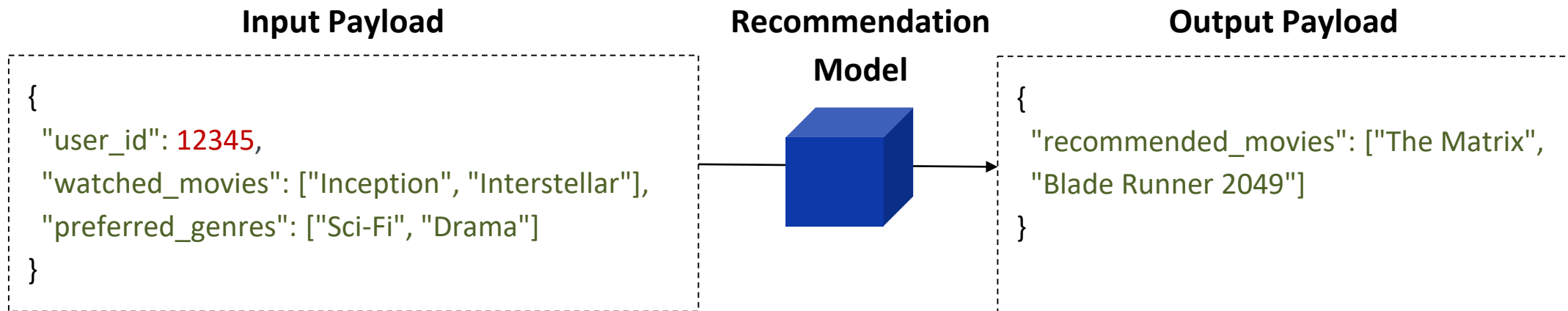
```
{  
  "user_id": 12345,  
  "watched_movies": ["Inception", "Interstellar"],  
  "preferred_genres": ["Sci-Fi", "Drama"]  
}
```

## Recommendation Model



# Payload: Example

- Let's consider an example of input and output payload for a content recommendation model:



# REST API Frameworks

- Need a framework to simplify the process of building and maintaining REST APIs:

# REST API Frameworks

- Need a framework to simplify the process of building and maintaining REST APIs:



# REST API Frameworks

- Need a framework to simplify the process of building and maintaining REST APIs:



- Common Python REST API frameworks in the industry:

**Flask**

**Building small to medium-sized APIs**



# REST API Frameworks

- Need a framework to simplify the process of building and maintaining REST APIs:



- Common Python REST API frameworks in the industry:

**Flask**

**Building small to medium-sized APIs**

**Fast API**

**Ideal for high-performance APIs**

# REST API Frameworks

- Need a framework to simplify the process of building and maintaining REST APIs:

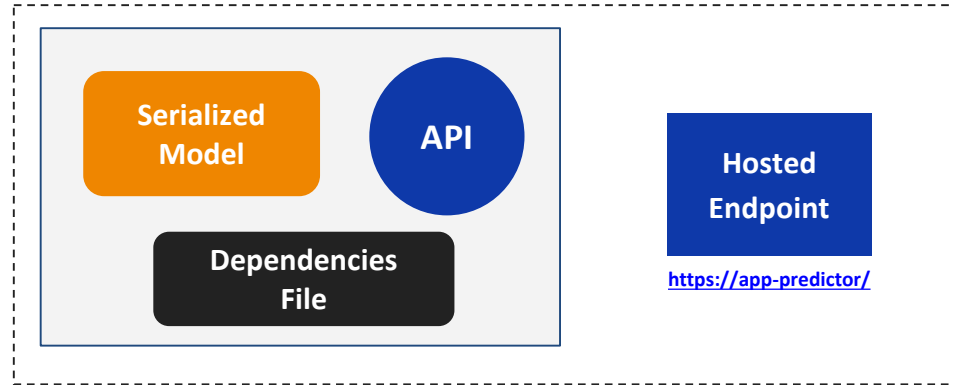


- Common Python REST API frameworks in the industry:

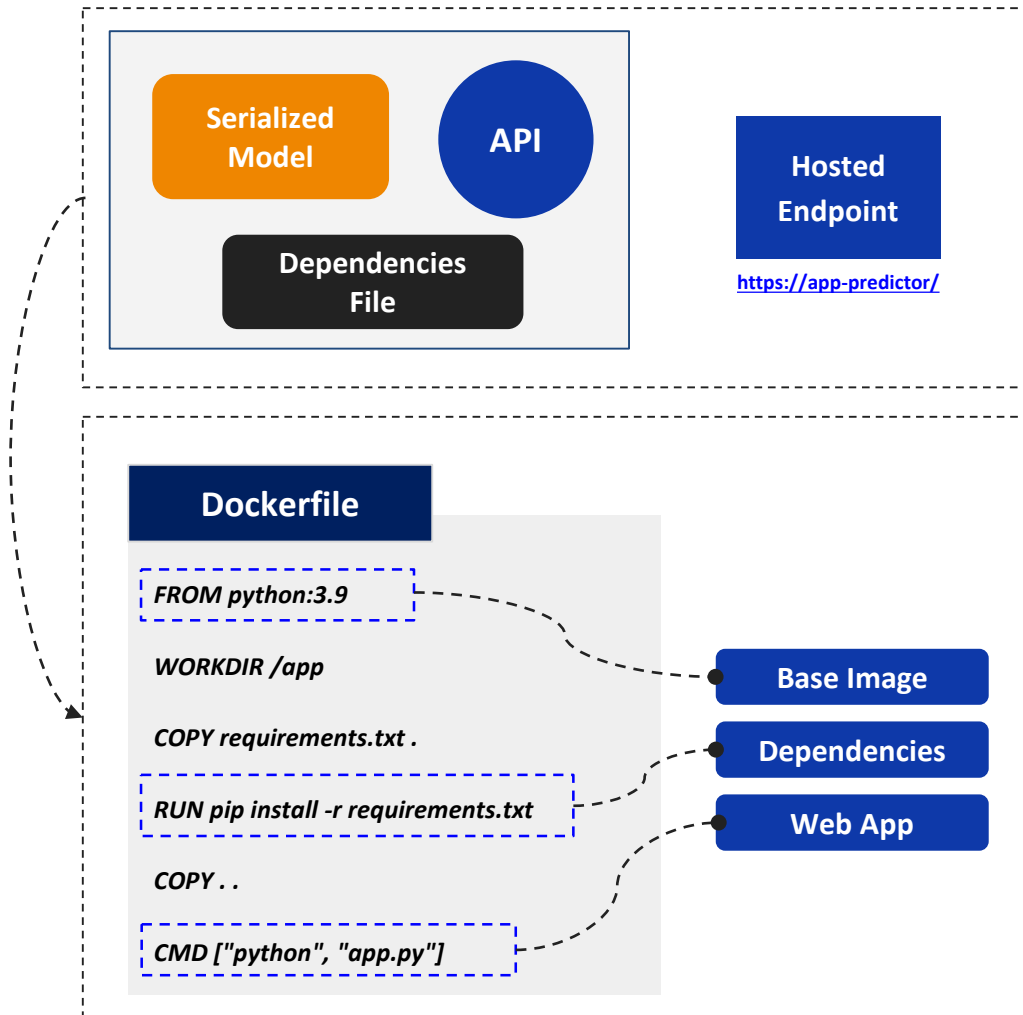
|          |                                     |
|----------|-------------------------------------|
| Flask    | Building small to medium-sized APIs |
| Fast API | Ideal for high-performance APIs     |
| Django   | Best for full-scale applications    |

# Video Break

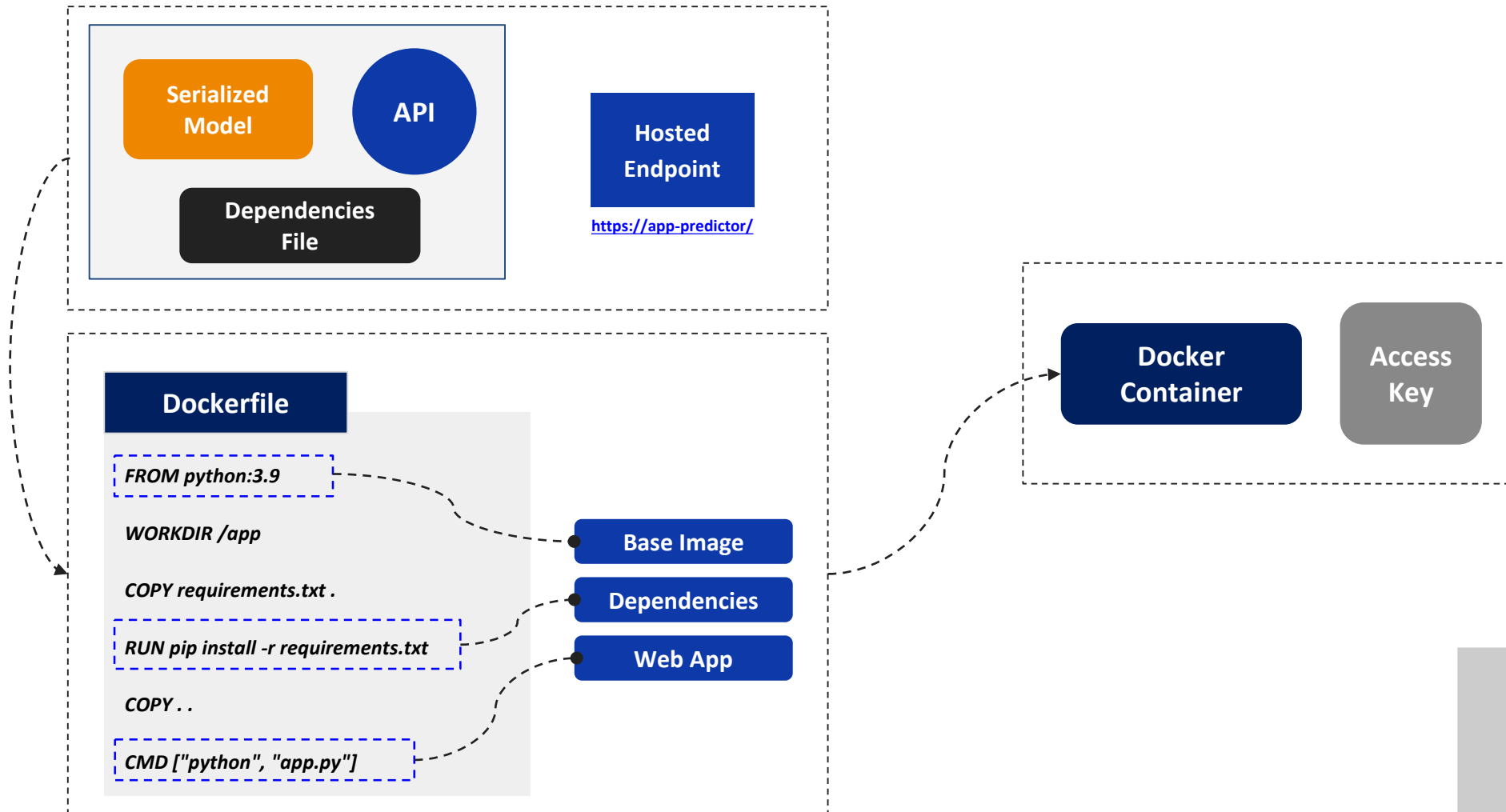
# Architecture of Containerized Model Deployment



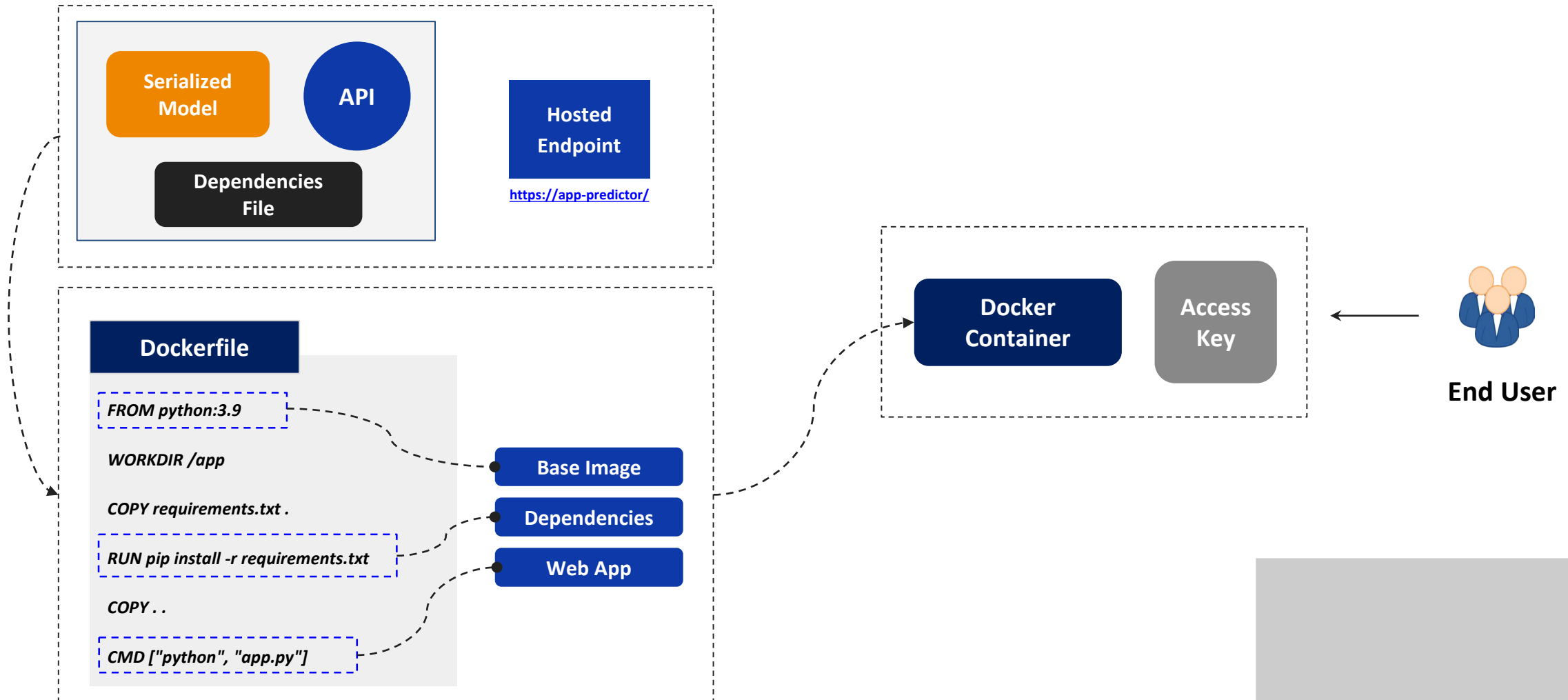
# Architecture of Containerized Model Deployment



# Architecture of Containerized Model Deployment



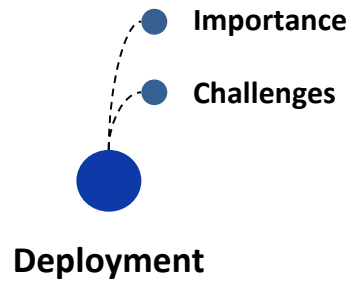
# Architecture of Containerized Model Deployment



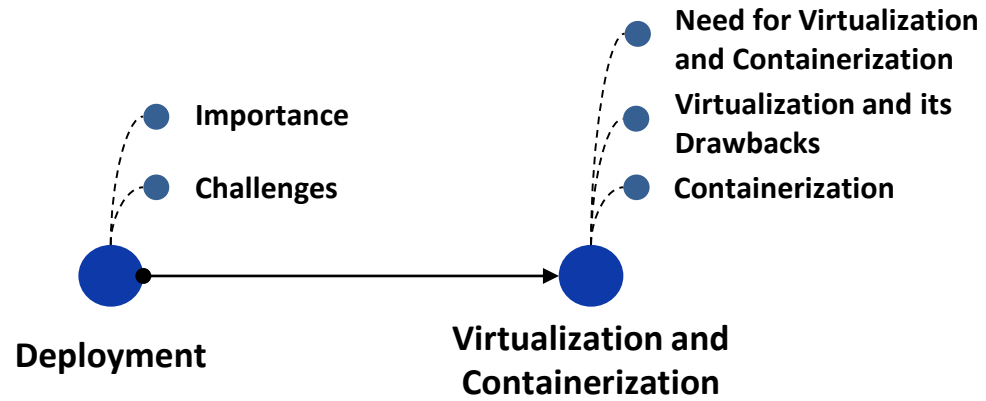
# Video Break



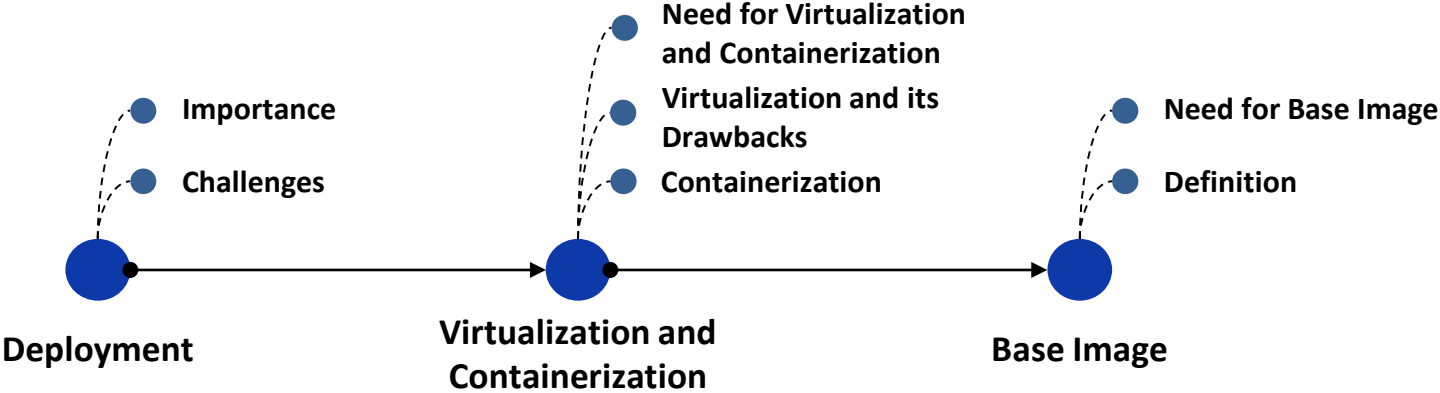
# Quick Recap



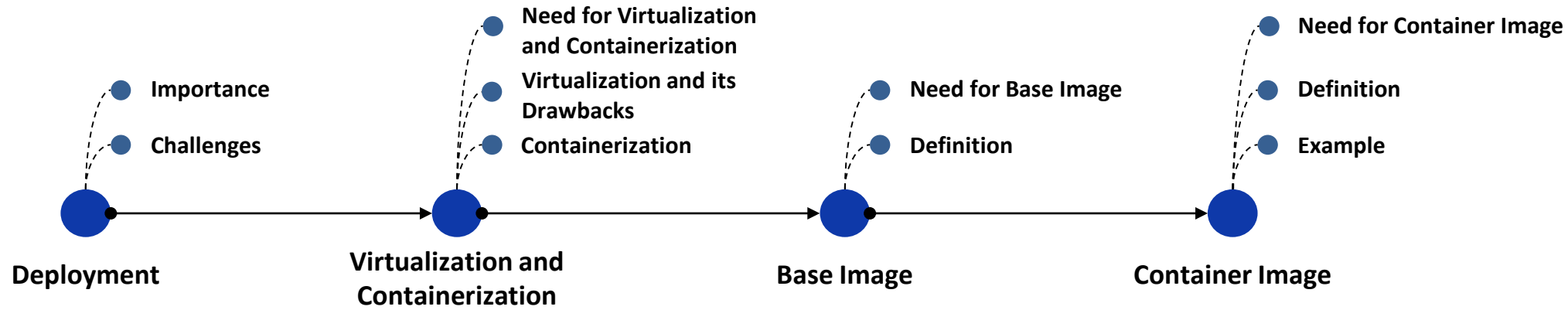
# Quick Recap



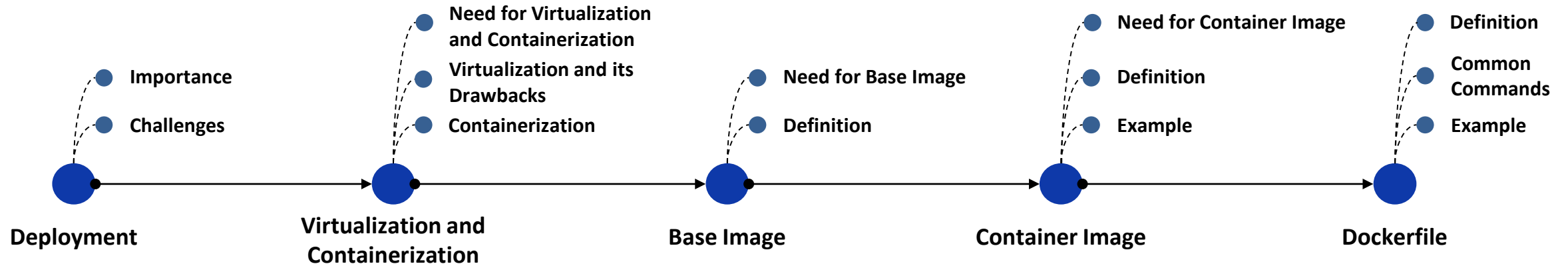
# Quick Recap



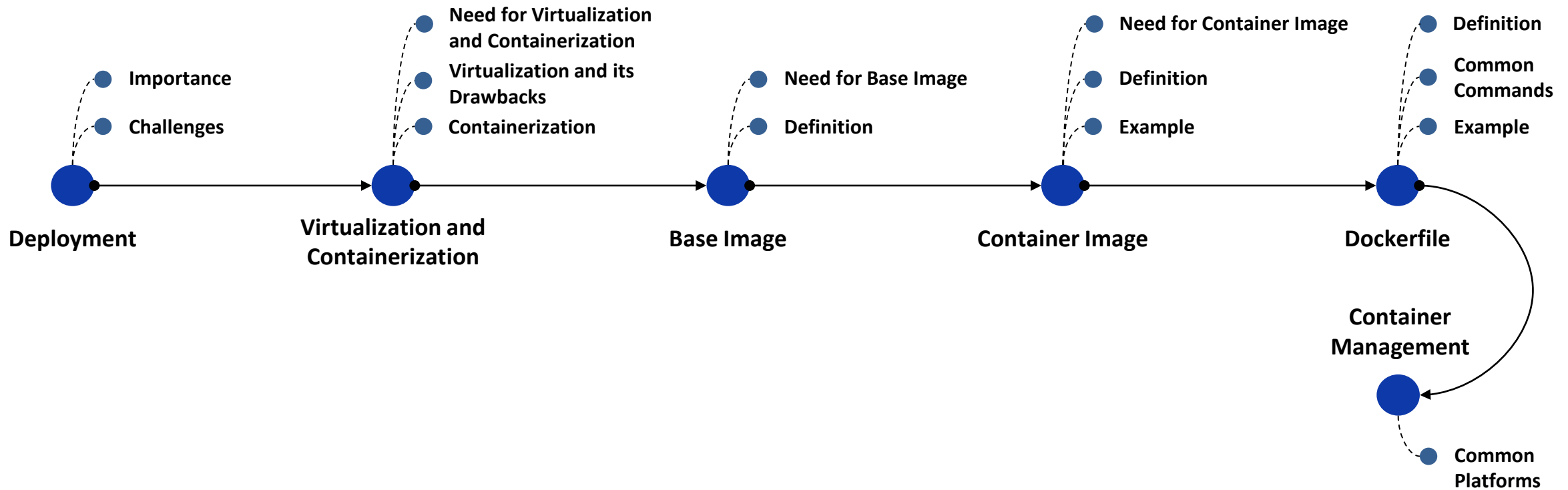
# Quick Recap



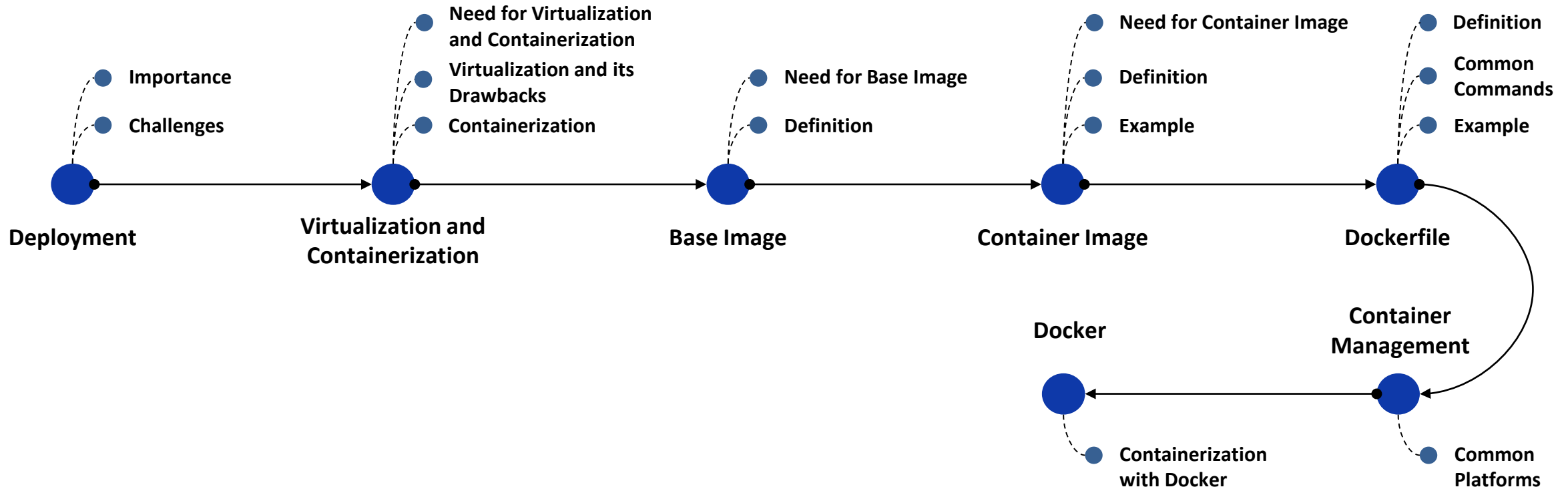
# Quick Recap



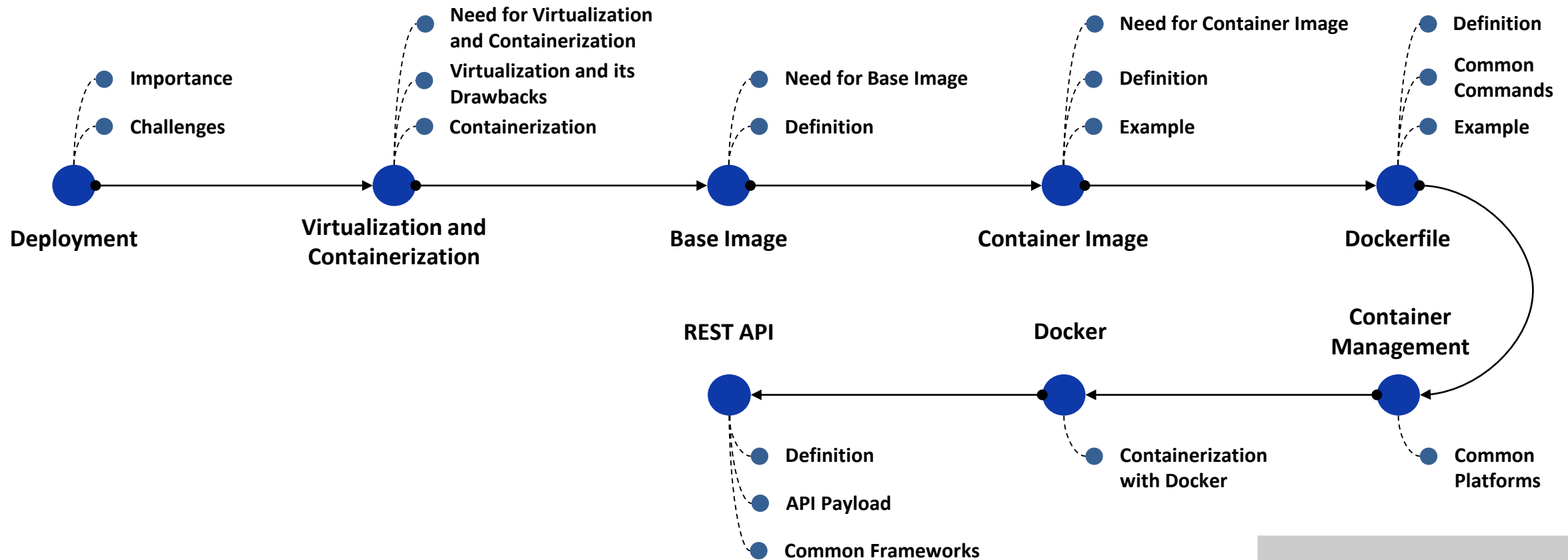
# Quick Recap



# Quick Recap

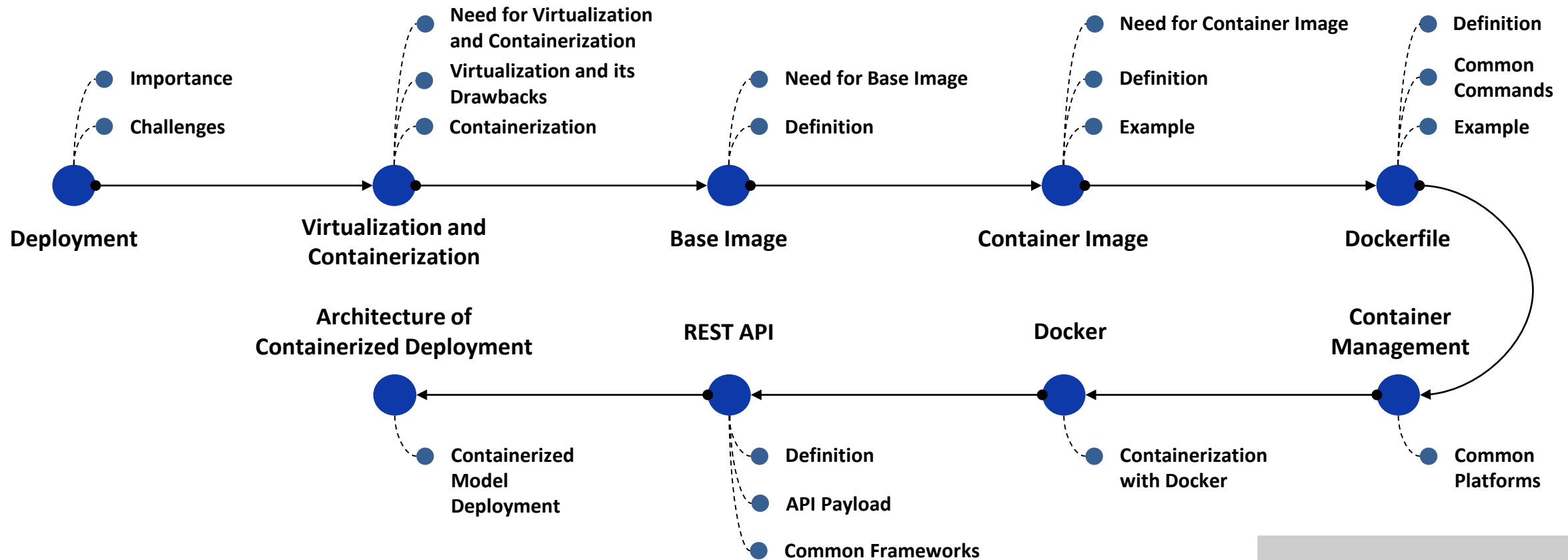


# Quick Recap





# Quick Recap



# Learning Outcomes

You should now be able to:

- Summarize the need for containerization in the context of model deployment and outline its advantages over virtualization.
- Examine the benefits of Docker for consistent deployments using containers.
- Design and implement a REST API to deploy a trained Machine Learning (ML) model.
- Deploy scalable, integrable, and manageable ML solutions using Docker containers and REST APIs.



# Power Ahead!

